
AWS FOR DEVELOPER & DEVOPS ENGINEER

Complete Guide with Real Scenarios, Tool Comparisons, and Interview Preparation

Version 1.0

About This Book

Dear Reader,

This book is designed for developers and DevOps engineers who want to master AWS services for their daily work and technical interviews.

What makes this book different:

- **Real scenarios:** Every service includes a practical example you'll actually use
- **Without cloud alternatives:** Understand what you're replacing
- **Tool comparisons:** EC2 vs Lambda vs ECS, S3 vs EBS vs EFS, RDS vs DynamoDB, ALB vs NLB
- **Advantages & disadvantages:** Make informed decisions
- **Diagrams:** Visualize every architecture
- **Interview questions:** Common questions with answers

By the end of this book, you will be able to:

- Choose the right AWS service for any use case
- Design scalable, resilient architectures on AWS
- Answer any AWS interview question confidently
- Implement DevOps practices on AWS

Let's begin your AWS journey.

[Page 2]

TABLE OF CONTENTS

PART ONE: AWS BASICS

- Chapter 1: What is AWS (Regions, AZs, Global Infrastructure)

PART TWO: COMPUTE SERVICES

- Chapter 2: EC2 (Elastic Compute Cloud)
- Chapter 3: Lambda (Serverless Functions)
- Chapter 4: ECS & EKS (Container Services)

PART THREE: STORAGE SERVICES

- Chapter 5: S3 (Simple Storage Service)
- Chapter 6: EBS (Elastic Block Store)
- Chapter 7: EFS (Elastic File System)

PART FOUR: NETWORKING SERVICES

- Chapter 8: VPC (Virtual Private Cloud)
- Chapter 9: Load Balancers (ALB vs NLB)
- Chapter 10: API Gateway
- Chapter 11: Route 53 (DNS)

PART FIVE: DATABASE SERVICES

- Chapter 12: RDS (Relational Database Service)
- Chapter 13: DynamoDB (NoSQL Database)

PART SIX: DEVOPS & CI/CD

- Chapter 14: CodePipeline, CodeBuild, CodeDeploy
- Chapter 15: CloudFormation & CDK (IaC)
- Chapter 16: CloudWatch (Monitoring & Logging)

PART SEVEN: SECURITY

- Chapter 17: IAM (Identity and Access Management)
- Chapter 18: Security Groups & NACLs
- Chapter 19: Secrets Manager & KMS

PART EIGHT: REAL SCENARIOS

- Chapter 20: Deploying a Web Application
- Chapter 21: Building a Serverless API
- Chapter 22: Event-Driven Image Processing

PART NINE: TOOL COMPARISONS (IMPORTANT)

- Chapter 23: EC2 vs Lambda vs ECS
- Chapter 24: S3 vs EBS vs EFS
- Chapter 25: RDS vs DynamoDB
- Chapter 26: ALB vs NLB

PART TEN: INTERVIEW PREPARATION

- Chapter 27: Common AWS Interview Questions
- Chapter 28: System Design with AWS
- Chapter 29: Troubleshooting Scenarios

[Page 3]

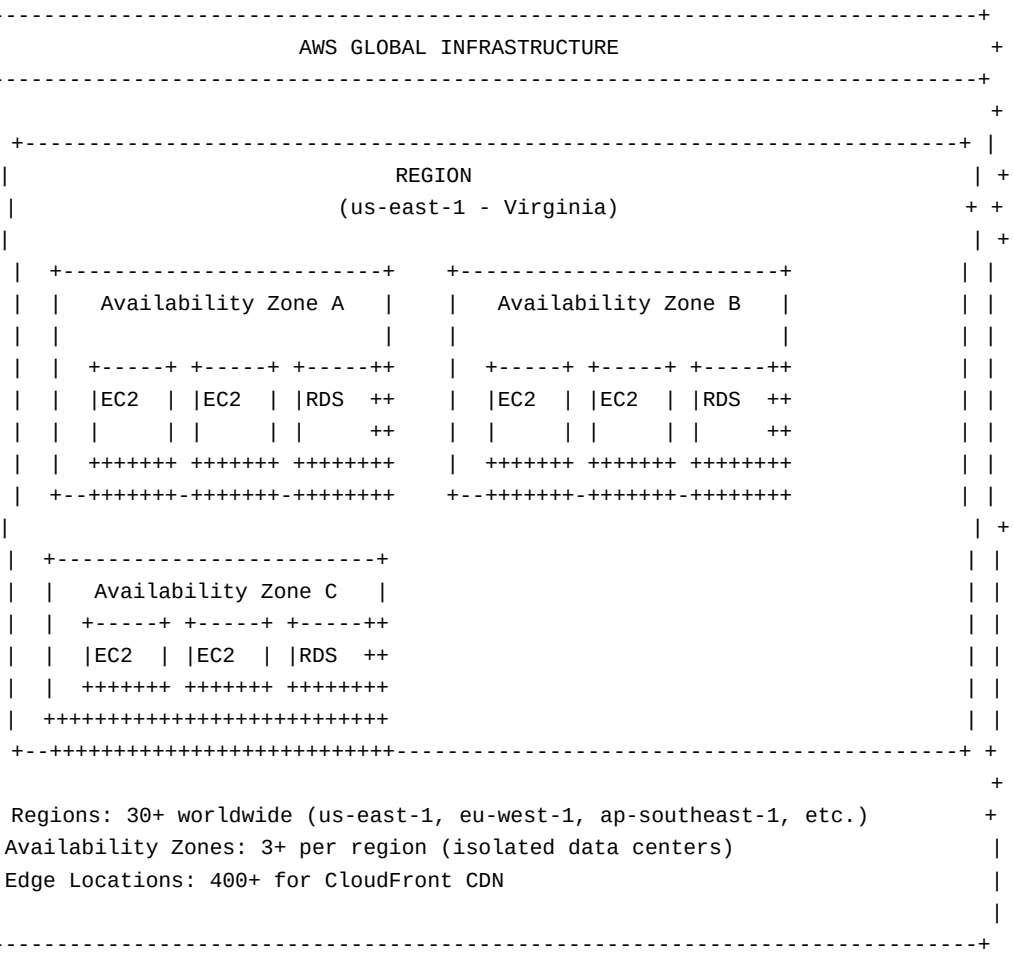
PART ONE: AWS BASICS

Chapter 1: What is AWS

Simple Definition

AWS (Amazon Web Services) is a cloud platform offering over 200 services including compute, storage, databases, networking, and machine learning—all available on demand over the internet.

Real Scenario
Your startup needs to host a website. You have two options:
- **Without AWS:** Buy servers ($10,000+), wait weeks, hire IT staff
- **With AWS:** Launch EC2 instance in 5 minutes, pay $0.042/hour
AWS Global Infrastructure
Diagram: AWS Global Infrastructure



Key Concepts
Concept
Description
Example
Why Important

Region
Geographic area
us-east-1 (Virginia), eu-west-1 (Ireland)
Latency, compliance, cost
Availability Zone (AZ)
One or more data centers
us-east-1a, us-east-1b
High availability (spread across AZs)
Edge Location
CDN endpoint
400+ globally
Low latency content delivery
Why Regions Matter

Factor	Explanation
--------	----------
Latency	Choose region closest to your users
Compliance	Data residency requirements (GDPR, etc.)
Cost	Prices vary by region (us-east-1 is cheapest)
Service Availability	New services launch in us-east-1 first

Global Services vs Regional Services

Type	Examples	Scope
------	----------	-------
Global	IAM, Route 53, CloudFront	Single configuration for all regions
Regional	EC2, S3, RDS, Lambda	Per-region resources

[Page 4]

PART TWO: COMPUTE SERVICES

Chapter 2: EC2 (Elastic Compute Cloud)

What is EC2?

Virtual servers in the cloud. You rent compute capacity (CPU, RAM, disk) by the hour or second.

Real Scenario

Your company needs to host a Node.js backend API that runs 24/7.

Without Cloud (Alternative):

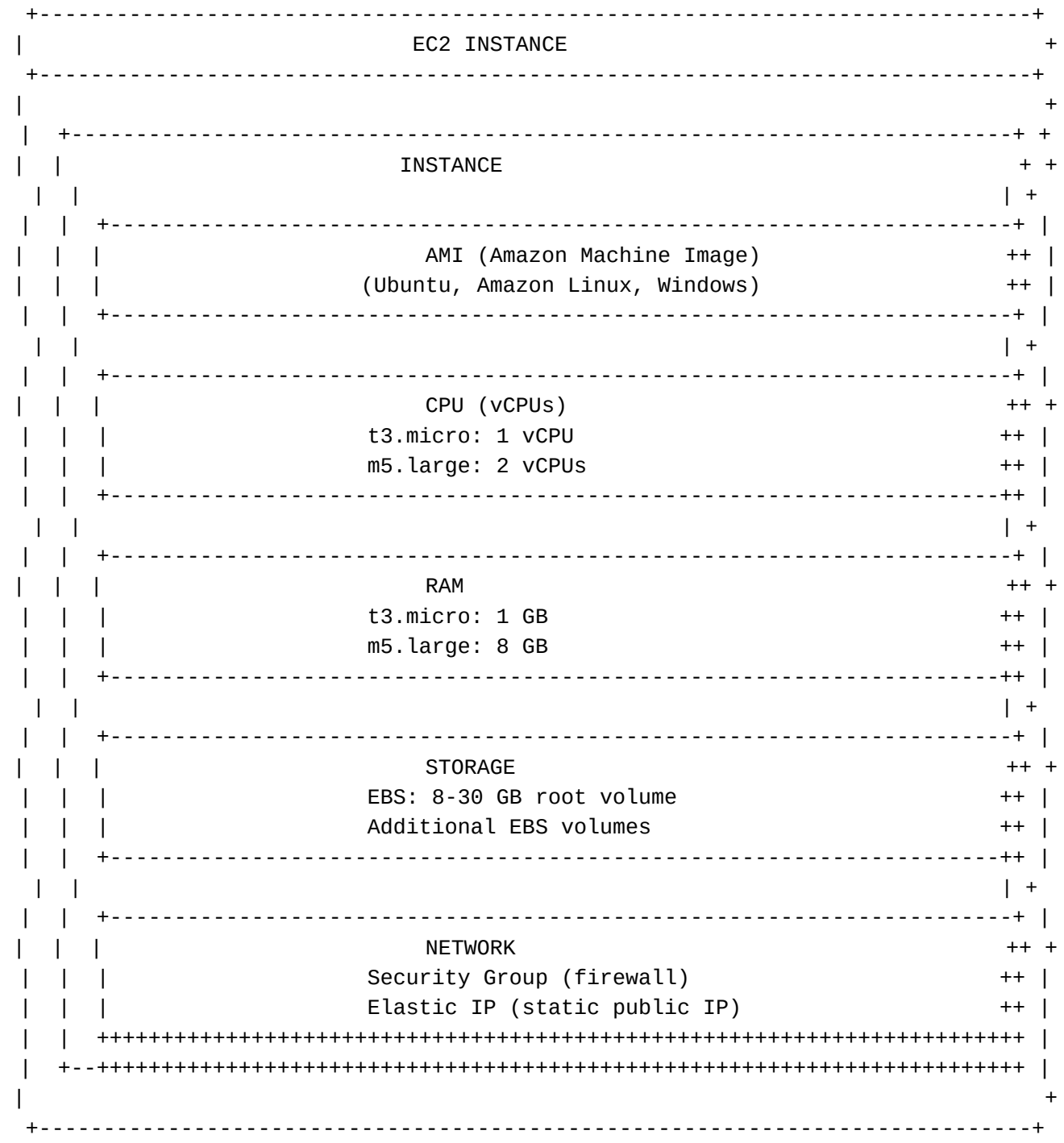
- Buy physical servers ($5,000+ upfront)
- Wait weeks for delivery
- Set up in a data center
- Manage hardware, cooling, power

- Hire IT staff
- Over-provision for peak or risk downtime

With EC2:

- Launch t3.micro instance (1 vCPU, 1 GB RAM) in 5 minutes
- Pay ~$0.0104/hour (~$7.50/month)
- Scale up to larger instances as traffic grows
- No hardware to manage

Diagram: EC2 Instance

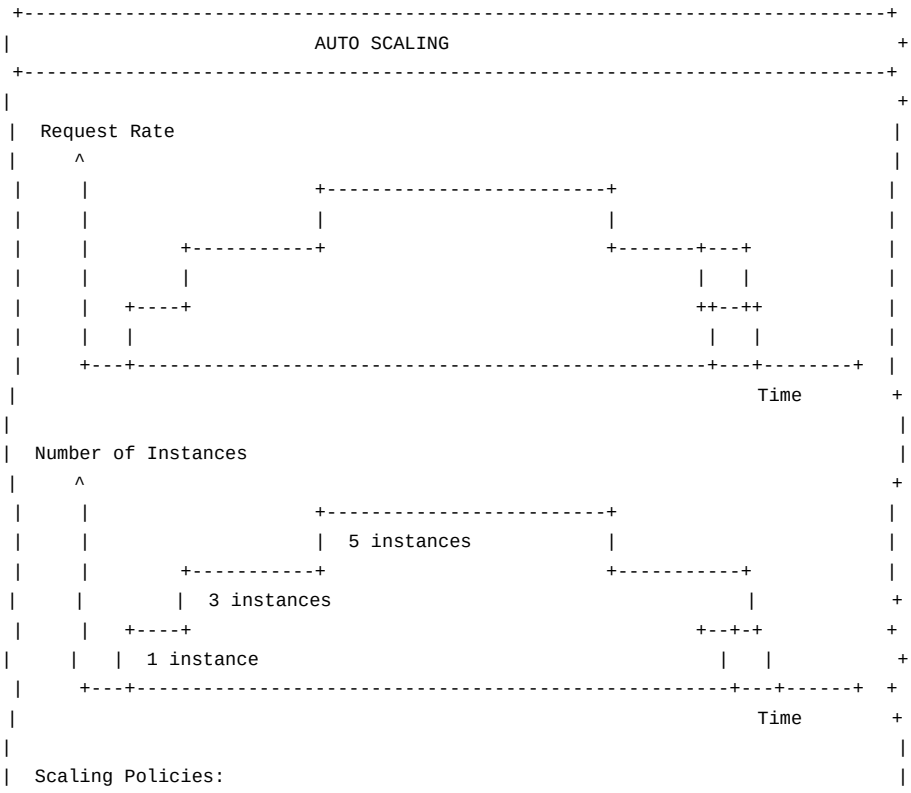


EC2 Instance Types (Families)

Family
Purpose
Example
Use Case

General Purpose (t, m)
Balanced
t3.micro, m5.large
Web servers, small DBs
Compute Optimized (c)
High CPU
c5.large
Batch processing, gaming
Memory Optimized (r, x)
High RAM
r5.large
In-memory DBs, caching
Storage Optimized (i, d)
High I/O
i3.large
NoSQL, data warehousing
GPU (g, p)
Graphics/ML
g4dn.xlarge
ML, video rendering
EC2 Pricing Models
Model
Description
Best for

On-Demand
Pay per hour/second
Unpredictable workloads
Reserved
1-3 year commitment (up to 75% off)
Steady-state workloads
Spot
Bid for unused capacity (up to 90% off)
Fault-tolerant, batch jobs
Savings Plans
Flexible commitment across families
Mixed workloads
EC2 Auto Scaling
Automatically adjusts number of EC2 instances based on demand.
Diagram: Auto Scaling



```
 |  - Scale out when CPU > 70% for 5 minutes                                |
 |  - Scale in when CPU < 30% for 10 minutes                                |
 |  - Scheduled scaling: 9 AM - 6 PM (business hours)                        +
|                                                                            +
 +--------------------------------------------------------------------------+
```

Common EC2 Commands (AWS CLI)

```
# Launch an EC2 instance
aws ec2 run-instances \
                            --image-id ami-0c55b159cbfafe1f0 \
                                --instance-type t3.micro \
                                  --key-name MyKeyPair \
                            --security-group-ids sg-903004f8 \
                                --subnet-id subnet-6e7f829e
# List instances
aws ec2 describe-instances
# Stop/Start/Terminate instance
aws ec2 stop-instances --instance-ids i-1234567890abcdef0
aws ec2 start-instances --instance-ids i-1234567890abcdef0
aws ec2 terminate-instances --instance-ids i-1234567890abcdef0
# SSH into instance
ssh -i MyKeyPair.pem ec2-user@54.123.45.67
```

When to Use EC2

Use Case	Why EC2
Lift-and-shift migration	Run existing apps without changes
Long-running workloads	24/7 applications
Custom software	Need specific OS or software
Legacy applications	Require full control
Batch processing	Can use Spot instances for savings

When NOT to Use EC2

Use Case	Better Option
Event-driven	Lambda (serverless)
Sporadic traffic	Lambda (pay per invocation)
Containers	ECS or EKS
Static websites	S3 + CloudFront

Advantages of EC2

1. **Full control:** Choose OS, software, configuration
2. **Wide variety:** 400+ instance types for any workload
3. **Flexible pricing:** On-demand, reserved, spot
4. **Integration:** Works with all AWS services
5. **Auto scaling:** Automatically adjust capacity

Disadvantages of EC2

1. **Management overhead:** You manage OS, patches, security
2. **Pay for idle:** Pay even when not at full utilization
3. **Manual scaling:** Need to configure auto scaling
4. **State management:** You manage data persistence

EC2 vs Other AWS Compute (Preview)

Aspect	EC2	Lambda	ECS
--------	-----	--------	-----
Control	Full	None	Moderate
Pricing	Per hour	Per invocation	Per hour
Startup	Minutes	Milliseconds	Seconds
State	Can be stateful	Stateless	Can be stateful
Max duration	Unlimited	15 minutes	Unlimited

(Detailed comparison in Chapter 23)

[Page 5]

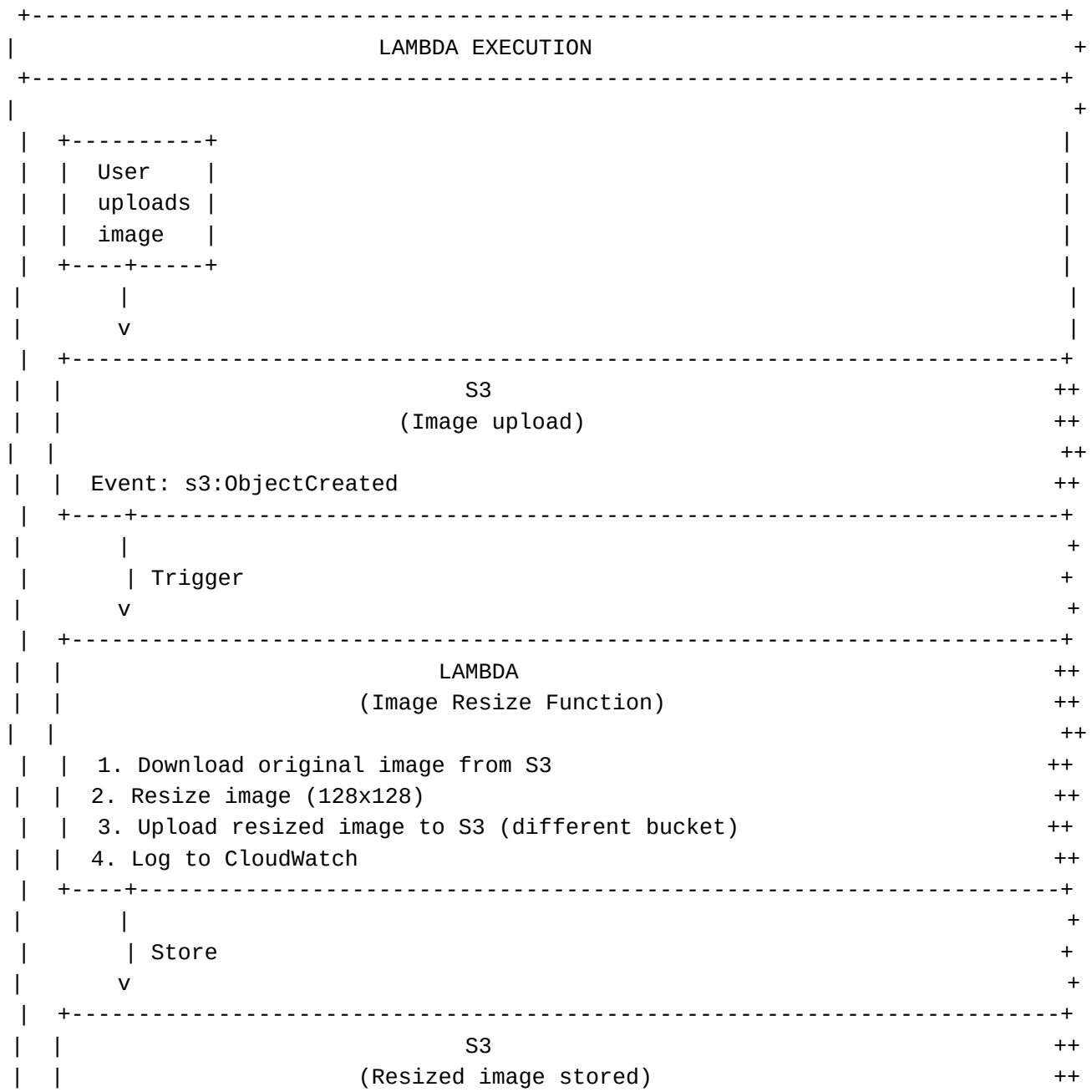
Chapter 3: Lambda (Serverless Functions)

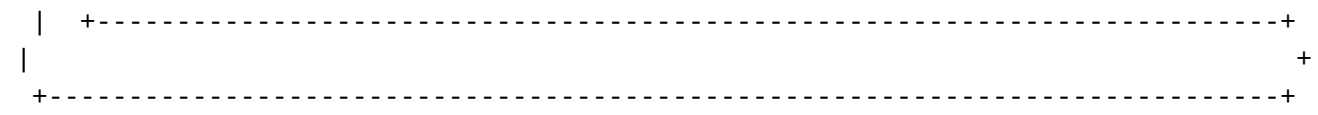
What is Lambda?

Run code without provisioning or managing servers. Upload your code, and Lambda runs it on demand. Pay only for compute time when your code runs.

Real Scenario

Your application needs to resize images whenever a user uploads a profile picture.
Without Cloud (Alternative):
- Run an EC2 instance 24/7 waiting for uploads
- Waste resources when no uploads happen (95% idle)
- Manage scaling during traffic spikes
- Handle server maintenance
With Lambda:
- Function triggers automatically on S3 upload
- Runs only when images are uploaded
- Scales automatically to thousands of concurrent executions
- Pay only for milliseconds of execution time
Diagram: Lambda Execution Flow





Lambda Sample Code (Node.js)

```
// Image resize Lambda function
const AWS = require('aws-sdk');
const sharp = require('sharp');
const s3 = new AWS.S3();
exports.handler = async (event) => {
// Get the uploaded image from S3 event
const bucket = event.Records[0].s3.bucket.name;
const key = decodeURIComponent(event.Records[0].s3.object.key);

// Download original image
const originalImage = await s3.getObject({ Bucket: bucket, Key: key }).promise();

// Resize image (128x128)
const resizedImage = await sharp(originalImage.Body)
.resize(128, 128)
.toBuffer();

// Upload resized image to different bucket
await s3.putObject({
                                    Bucket: 'my-resized-images',
Key: `resized/${key}`,
Body: resizedImage,
ContentType: 'image/jpeg'
}).promise();

return {
statusCode: 200,
body: JSON.stringify({ message: 'Image resized successfully' })
};
};
```

Lambda Triggers (Event Sources)

Trigger	Use Case
API Gateway	REST API backend
S3	File processing on upload
DynamoDB Streams	React to database changes
SQS	Process queue messages
SNS	Handle notifications
CloudWatch Events	Scheduled jobs (cron)
Kinesis	Stream processing

Lambda Limits

Limit	Value
-------	-------
Timeout	15 minutes maximum
Memory	128 MB - 10 GB
Temporary disk (/tmp)	512 MB - 10 GB
Request size	6 MB (sync), 256 KB (async)
Concurrent executions	1,000 (soft limit, can increase)
Function size	50 MB (zipped), 250 MB (unzipped)

Lambda Pricing (as of 2024)

- **Compute:** $0.0000166667 per GB-second
- **Requests:** $0.20 per 1 million requests
- **Free tier:** 1 million requests/month + 400,000 GB-seconds

Example pricing:

- 1 million requests @ 128 MB for 100ms = ~$0.20 (requests) + $0.21 (compute) = $0.41

When to Use Lambda

Use Case	Why Lambda
----------	----------
Event-driven processing	Respond to events (S3, DynamoDB)
API backends	With API Gateway
File processing	Images, videos, documents
Data transformation	ETL pipelines
Scheduled tasks	CloudWatch Events (cron)
Chatbots	Alexa skills, Slack bots

When NOT to Use Lambda

Use Case	Better Option
----------	----------
Long-running processes (>15 min)	EC2 or ECS
Stateful applications	EC2
Low-latency (<10ms) requirements	EC2 (warm)
Compute-intensive workloads	EC2 (GPU instances)
Large deployments	EC2 or containers

Advantages of Lambda

1. **No server management:** Focus on code, not infrastructure
2. **Automatic scaling:** From 0 to thousands of concurrent executions
3. **Pay per use:** No cost when not running
4. **Built-in integrations:** S3, DynamoDB, API Gateway, etc.
5. **Multi-language support:** Node.js, Python, Java, Go, .NET, Ruby

Disadvantages of Lambda

1. **Cold starts:** First invocation can be slow (100ms-1s)
2. **Timeout limit:** 15 minutes maximum
3. **Stateless:** No local state between invocations
4. **Debugging complexity:** Harder than EC2
5. **Vendor lock-in:** Lambda-specific features

Lambda vs EC2 (Preview)

Aspect	Lambda	EC2
--------	--------	-----
Pricing	Per invocation	Per hour (even idle)
Scaling	Automatic	Manual/Auto Scaling
Cold start	Yes (100ms-1s)	No
Control	None	Full
Best for	Event-driven, sporadic	Long-running, custom

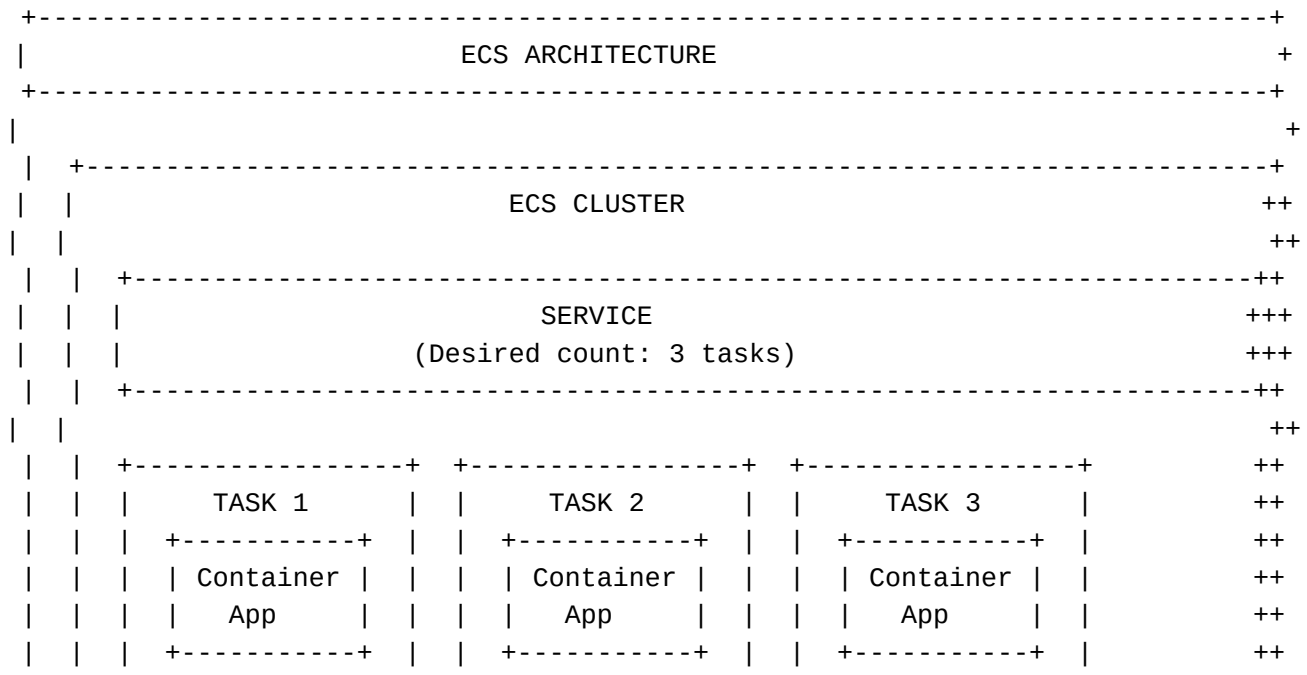
(Detailed comparison in Chapter 23)

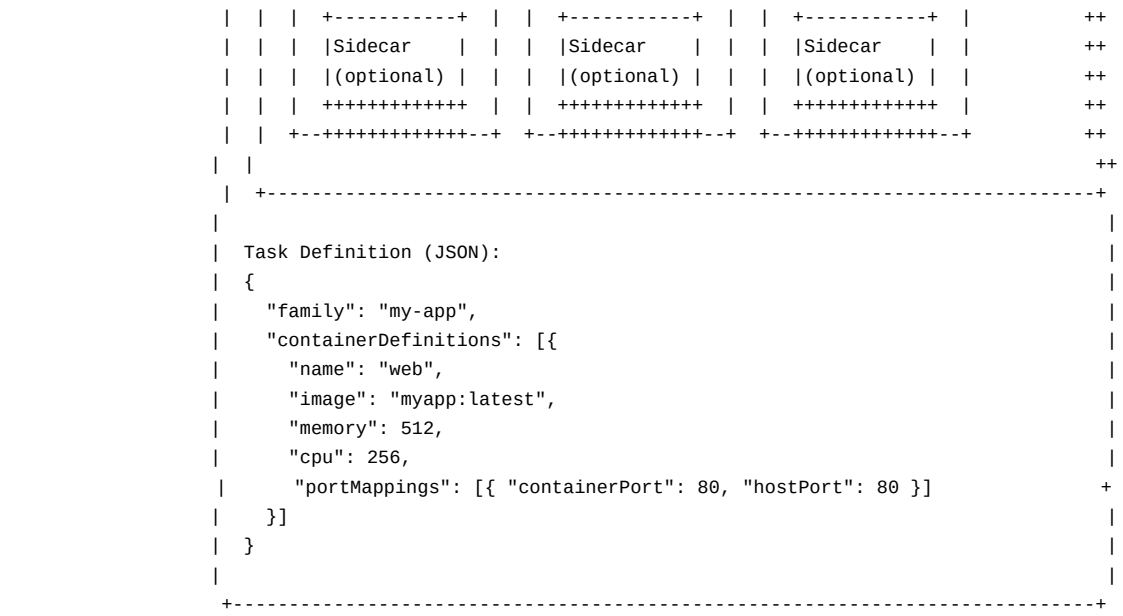
[Page 6]

Chapter 4: ECS & EKS (Container Services)

What is ECS?

Elastic Container Service is AWS's native container orchestration service. Run and scale Docker containers on AWS.
What is EKS?
Elastic Kubernetes Service is managed Kubernetes on AWS. Run Kubernetes without managing the control plane.
Real Scenario
Your team has containerized a microservices application (Docker images). You need to run and scale these containers.
Without Cloud (Alternative):
- Set up Kubernetes cluster (weeks of work)
- Manage control plane (etcd, API server, scheduler)
- Handle upgrades, patches, security
- Scale infrastructure yourself
With ECS/EKS:
- Deploy containers in minutes
- AWS manages control plane (EKS) or simpler (ECS)
- Automatic scaling of containers
- Integrated with AWS services (ALB, IAM, CloudWatch)
Diagram: ECS Architecture





Launch Types

Launch Type	Description	Best for
Fargate	Serverless containers (no EC2 management)	Simple, no infrastructure concerns
EC2	Run containers on your EC2 instances	Cost optimization, custom AMIs

ECS vs EKS Comparison

Aspect	ECS	EKS
Type	AWS native	Kubernetes (open source)
Complexity	Simpler	More complex
Control plane	Managed by AWS	Managed by AWS (EKS)
Portability	AWS only	Multi-cloud (K8s)
Learning curve	Lower	Higher
Community	Smaller	Large (open source)
Tooling	AWS tools	kubectl, Helm, Istio
When to use	AWS-only, simpler needs	Portability, K8s expertise

When to Use ECS

Use Case	Why ECS
AWS-only environment	No need for portability

Use Case	Why ECS
Simplicity	Easier than Kubernetes
Deep AWS integration	IAM, ALB, CloudWatch
Serverless containers	Fargate launch type

When to Use EKS

Use Case	Why EKS
----------	---------
Multi-cloud strategy	Run same K8s on AWS, GCP, Azure
Existing Kubernetes expertise	Team already knows K8s
Open source tooling	Helm, Istio, Prometheus
Hybrid deployments	On-prem + cloud K8s

Advantages of ECS

1. **Simpler:** Easier to learn and operate
2. **Native integration:** Works seamlessly with AWS services
3. **Fargate option:** No EC2 management
4. **Fast deployment:** Quick to set up

Disadvantages of ECS

1. **Vendor lock-in:** AWS only
2. **Smaller ecosystem:** Fewer third-party tools
3. **Less community support**

Advantages of EKS

1. **Portability:** Run on any cloud or on-premises
2. **Large ecosystem:** Helm, Istio, Prometheus, Grafana
3. **Community support:** Large open source community
4. **Flexibility:** Customizable with CRDs and operators

Disadvantages of EKS

1. **Complexity:** Steeper learning curve

2. **Management overhead:** Still need to manage worker nodes
3. **Cost:** $0.10 per hour for control plane

[Page 7]

PART THREE: STORAGE SERVICES

Chapter 5: S3 (Simple Storage Service)

What is S3?

Object storage for files, images, videos, backups, and static websites. Unlimited storage, pay per GB stored and per request.

Real Scenario

Your application needs to store user profile pictures and product images.

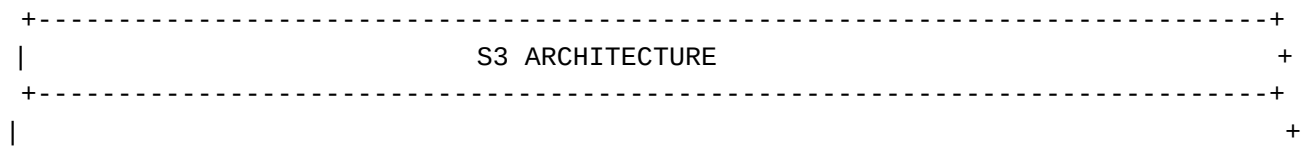
Without Cloud (Alternative):

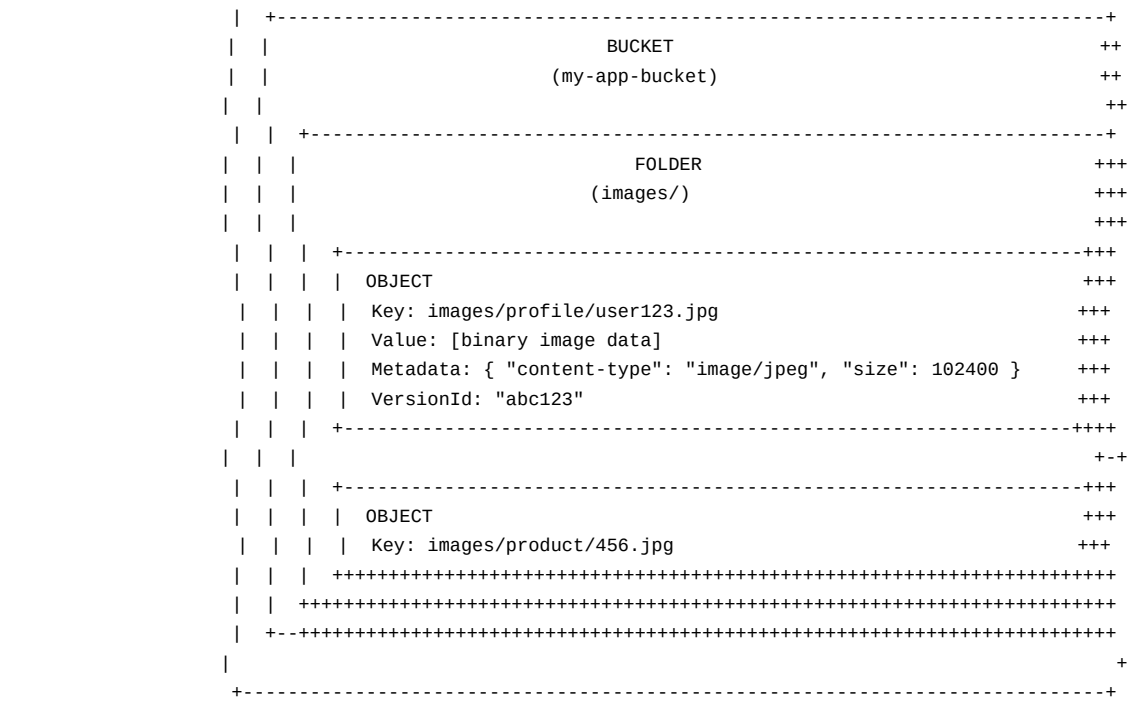
- Set up file servers
- Manage disk space
- Implement backup and replication
- Handle scaling and performance

With S3:

- Create a bucket in minutes
- Upload files via API or console
- Automatic 11 nines durability
- Pay only for what you store

Diagram: S3 Architecture





S3 Storage Classes

Class
Use Case
Durability
Availability
Retrieval

S3 Standard
Frequent access
99.999999999%
99.99%
Immediate
S3 Intelligent-Tiering
Unknown access
99.999999999%
99.9%
Immediate
S3 Standard-IA
Infrequent access
99.999999999%
99.9%
Immediate
S3 One Zone-IA
Recreatable data
99.999999999%
99.5%
Immediate
S3 Glacier
Archives
99.999999999%
—
Minutes-hours
S3 Glacier Deep Archive
Long-term
99.999999999%
—
Hours

S3 Features

Feature
Description

Versioning
Keep multiple versions of an object
Lifecycle policies
Auto-transition to cheaper classes
Encryption

S3 API Example (AWS CLI)

```
# Create bucket
aws s3 mb s3://my-app-bucket

# Upload file
aws s3 cp image.jpg s3://my-app-bucket/images/

# Download file
aws s3 cp s3://my-app-bucket/images/image.jpg ./download.jpg

# Sync directory
aws s3 sync ./local-folder s3://my-app-bucket/folder/

# Set bucket policy (public read for static website)
aws s3api put-bucket-policy --bucket my-app-bucket --policy '{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-app-bucket/*"
  }]
}'
```

When to Use S3

Use Case	**Why S3**
----------	--------
Static websites	HTML, CSS, JS files
User uploads	Profile pictures, documents
Backups	Database backups, logs
Data lakes	Analytics, ML training data
Media storage	Images, videos
Distributing software	Installers, packages

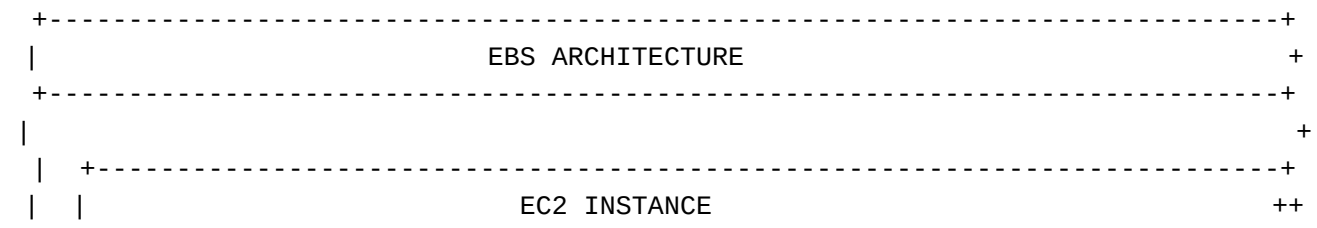
Advantages of S3

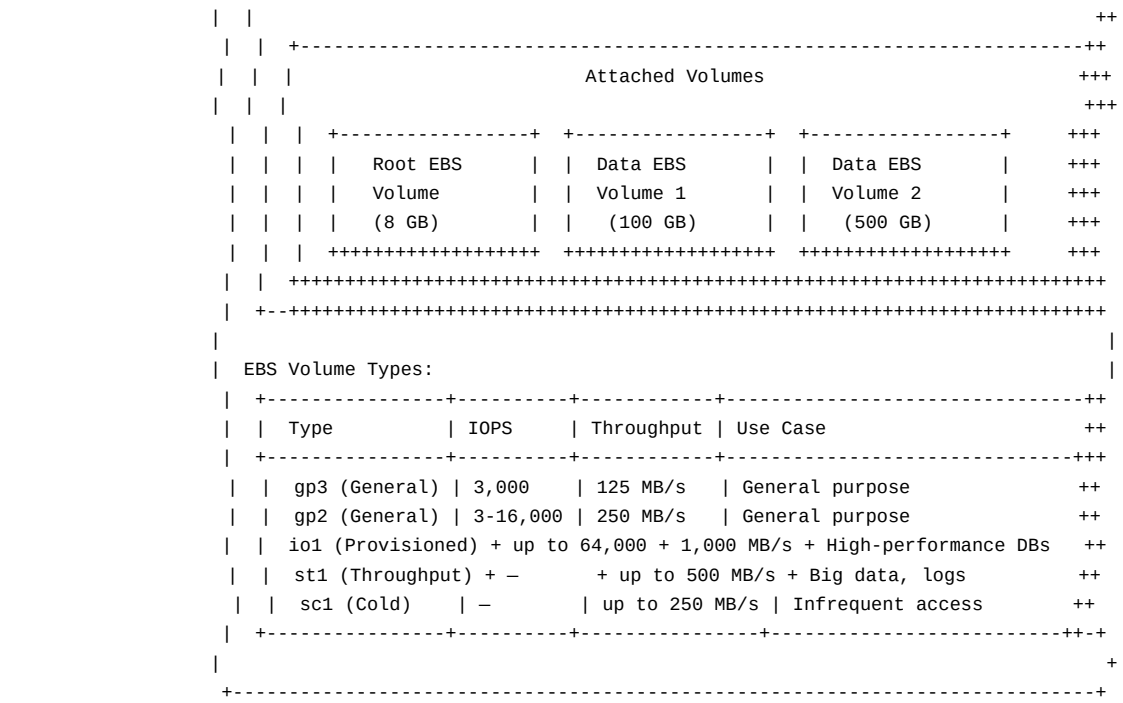
1. **11 nines durability:** 99.999999999% (you won't lose data)
2. **Unlimited storage:** No capacity planning
3. **Pay per use:** No minimum fees
4. **Global namespace:** Bucket names are globally unique
5. **Integration:** Works with all AWS services

Disadvantages of S3

1. **Eventual consistency:** For overwrites and deletes

2. **No file system semantics:** Can't mount as a drive
3. **Request costs:** GET, PUT requests cost money
4. **Not for live files:** Not suitable for frequently updating files
---
[Page 8]
Chapter 6: EBS (Elastic Block Store)
What is EBS?
Block storage for EC2 instances. Like a hard drive you attach to your EC2 instance. Can only be attached to one instance at a time.
Real Scenario
Your EC2 instance needs high-performance, low-latency storage for a database.
Without Cloud (Alternative):
- Install physical hard drive in server
- Limited capacity (add more drives)
- No backup automation
- Manual replacement on failure
With EBS:
- Create EBS volume in seconds
- Attach to EC2 instance
- 1 GB to 16 TB per volume
- Automatic backups (snapshots)
Diagram: EBS Architecture





EBS Features
Feature
Description

Snapshots
Point-in-time backups to S3
Encryption
AES-256 encryption at rest
Multi-attach
io1/io2 can attach to multiple instances (limited)
Elastic volumes
Modify size, type, IOPS without downtime
EBS vs Instance Store
Aspect
EBS
Instance Store

Persistence
Data persists after stop/terminate
Data lost on stop
Backup
Snapshots
No built-in backup
Performance
High
Very high (local NVMe)
Use case
Databases, boot volumes
Temporary data, cache
When to Use EBS
Use Case
Why EBS

Boot volumes
Root disk for EC2

Use Case	Why EBS
Databases	MySQL, PostgreSQL, MongoDB
Application data	Files that need persistence
High IOPS workloads	Provisioned IOPS (io1/io2)

Advantages of EBS

1. **Persistent storage:** Data survives EC2 stop/start
2. **Snapshots:** Easy backups to S3
3. **Elastic:** Resize without downtime
4. **Encryption:** Built-in encryption
5. **Performance:** High IOPS and throughput

Disadvantages of EBS

1. **Single attachment:** Can't share across instances (except multi-attach)
2. **Cost:** More expensive than S3 per GB
3. **AZ bound:** Can't move across availability zones

[Page 9]

Chapter 7: EFS (Elastic File System)

What is EFS?

Fully managed file storage that can be shared across multiple EC2 instances (like a network drive).

Real Scenario

Multiple web servers need access to shared content files (images, configuration, logs).

Without Cloud (Alternative):

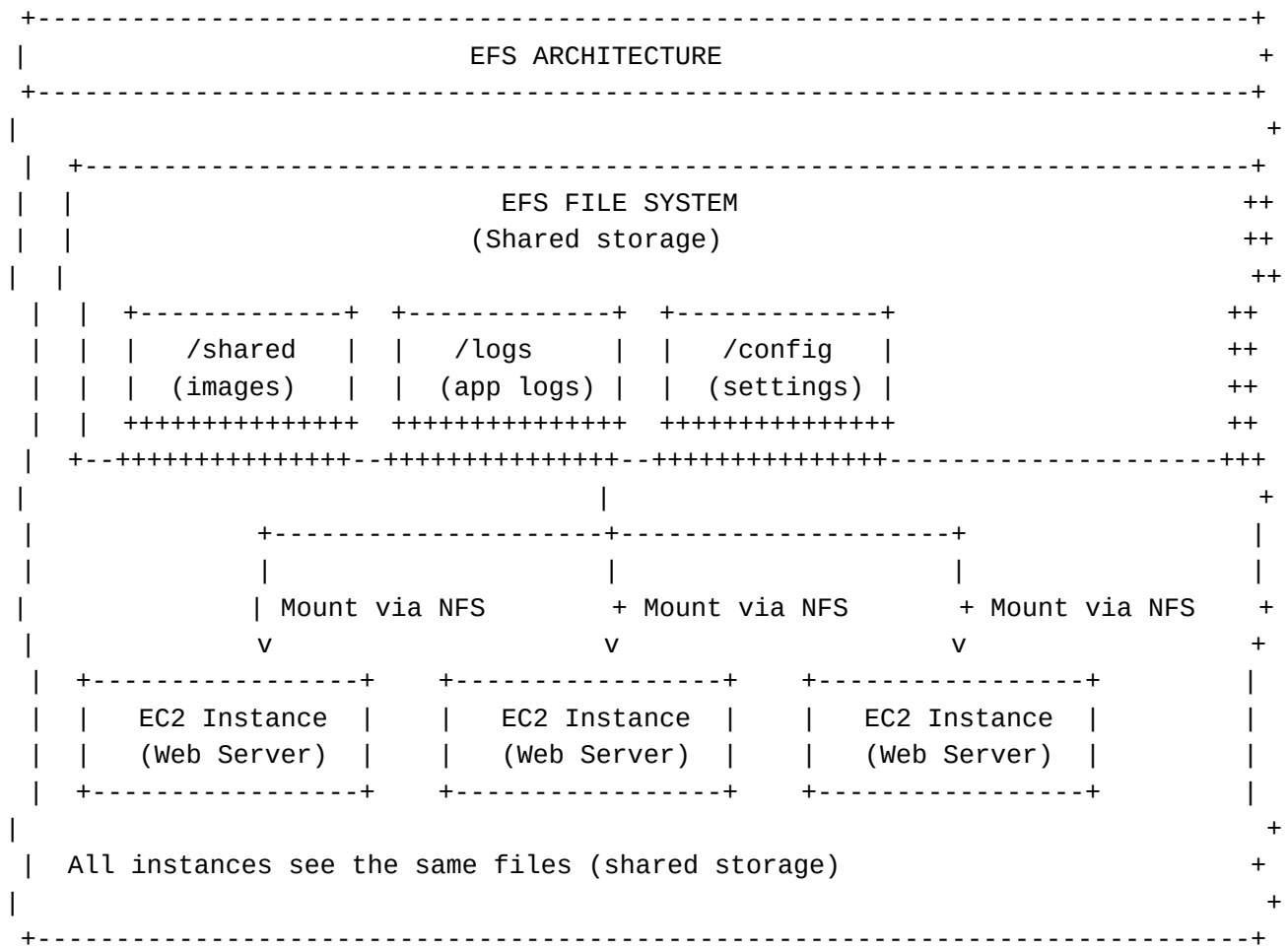
- Set up NFS server

- Manage storage capacity
- Handle backup and replication
- Configure security

With EFS:

- Create file system in minutes
- Automatically scales storage up/down
- Can be mounted on multiple EC2 instances
- Pay only for storage used

Diagram: EFS Architecture



EFS Performance Modes

Mode
Description
Use Case

General Purpose
Lower latency
Web servers, content management
Max I/O
Higher throughput
Big data, analytics

EFS Throughput Modes

Mode	Description
------	----------
Bursting	Baseline + burst credits (like T2 EC2)
Provisioned	Set throughput independently of storage

When to Use EFS

Use Case	Why EFS
----------	---------
Shared content	Multiple servers need same files
Web server farms	Serve same content from all nodes
Development environments	Share code across instances
Container storage	Persistent storage for containers

Advantages of EFS

1. **Shared access:** Multiple EC2 instances can mount same file system
2. **Automatic scaling:** Grows/shrinks with data
3. **Fully managed:** No patching, backups handled
4. **POSIX-compliant:** Works with standard file APIs
5. **Pay per use:** No minimum size

Disadvantages of EFS

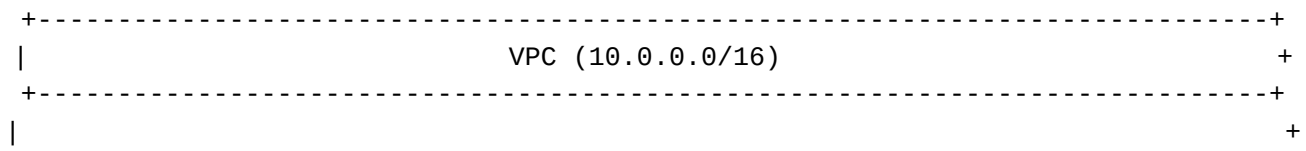
1. **Higher latency:** Slower than EBS (network attached)
2. **Cost:** More expensive than EBS per GB
3. **Performance:** Lower IOPS than Provisioned IOPS EBS

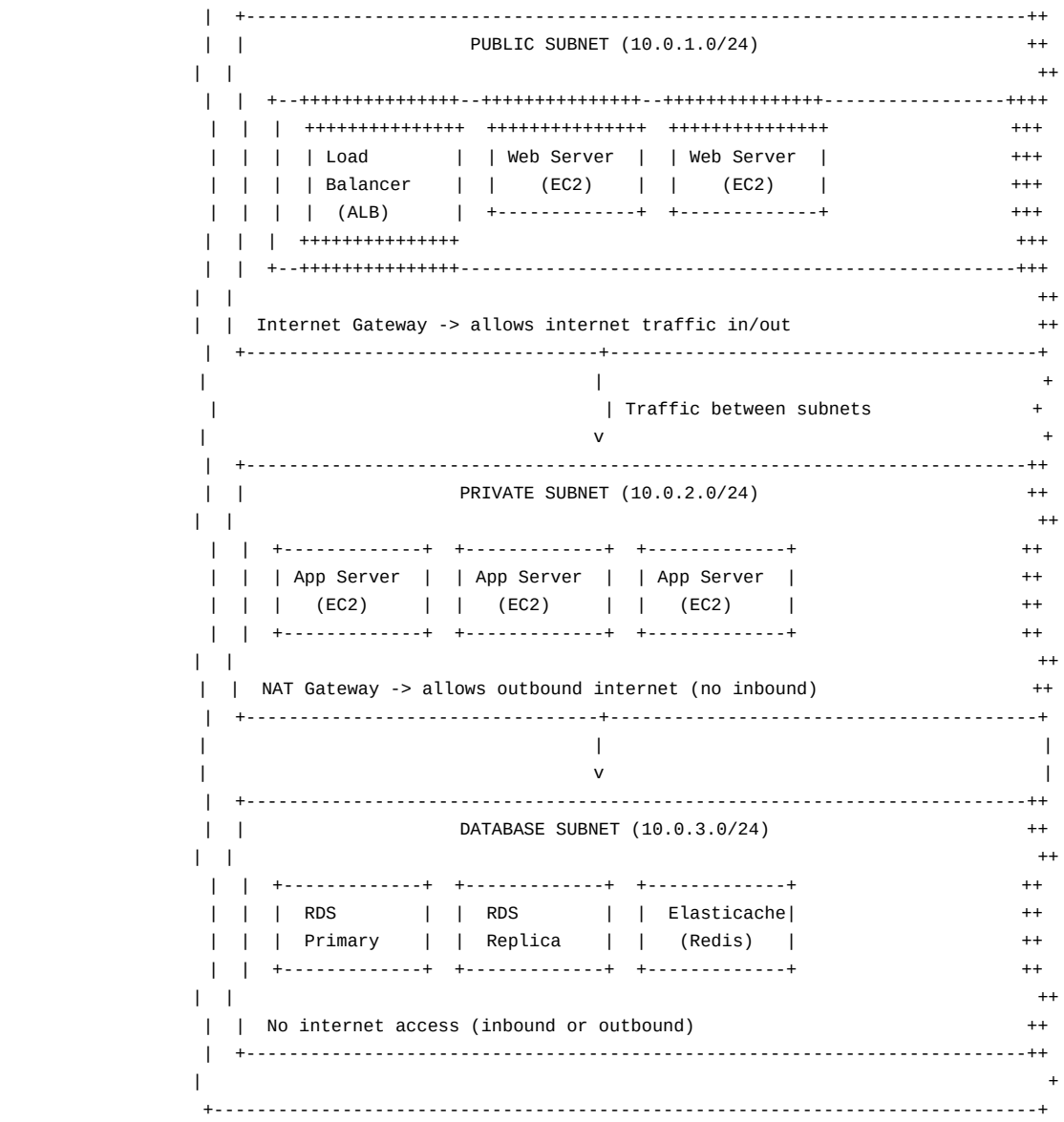
EBS vs EFS (Preview)

Aspect	EBS	EFS
--------	-----	-----
Access	Single EC2	Multiple EC2 (NFS)
Latency	Very low	Moderate
IOPS	High	Lower
Pricing	Provisioned capacity	Pay for used storage
Use case	Databases, boot	Shared files, logs

(Detailed comparison in Chapter 24)

[Page 10]
PART FOUR: NETWORKING SERVICES
Chapter 8: VPC (Virtual Private Cloud)
What is VPC?
A logically isolated network in AWS where you launch resources. You control IP ranges, subnets, route tables, and gateways.
Real Scenario
Your application needs separate networks for web servers (public internet access) and databases (no internet access).
Without Cloud (Alternative):
- Buy routers, switches, firewalls
- Configure physical networking
- Set up VPNs for secure access
- Manage all hardware
With VPC:
- Define IP range (10.0.0.0/16)
- Create public and private subnets
- Configure route tables and security groups
- All in software, minutes to create
Diagram: VPC with Public and Private Subnets





VPC Components

Component	Description	Purpose
CIDR Block	IP range (e.g., 10.0.0.0/16)	Defines VPC address space
Subnet	Range of IPs within VPC	Isolate resources
Internet Gateway	Connects VPC to internet	Public internet access
NAT Gateway	Private subnet -> internet	Software updates, APIs
Route Table	Rules for traffic routing	Direct traffic
Security Group	Instance-level firewall	Allow/deny traffic
NACL	Subnet-level firewall	Additional security layer
VPC Peering	Connect two VPCs	Cross-VPC communication
VPN Gateway	Connect to on-premises	

Security Groups vs NACLs

Aspect	Security Group	NACL (Network ACL)
--------	----------	----------
Level	Instance	Subnet
State	Stateful (return allowed)	Stateless (rules both ways)
Rules	Only Allow	Allow and Deny
Order	All evaluated	Rules in order (first match)
Use case	Instance security	Subnet security

When to Use VPC

Use Case	Why VPC
----------	---------
Any AWS deployment	Required for EC2, RDS, etc.
Multi-tier apps	Separate web, app, DB layers
Security compliance	Isolated network
Hybrid cloud	VPN to on-premises

Advantages of VPC

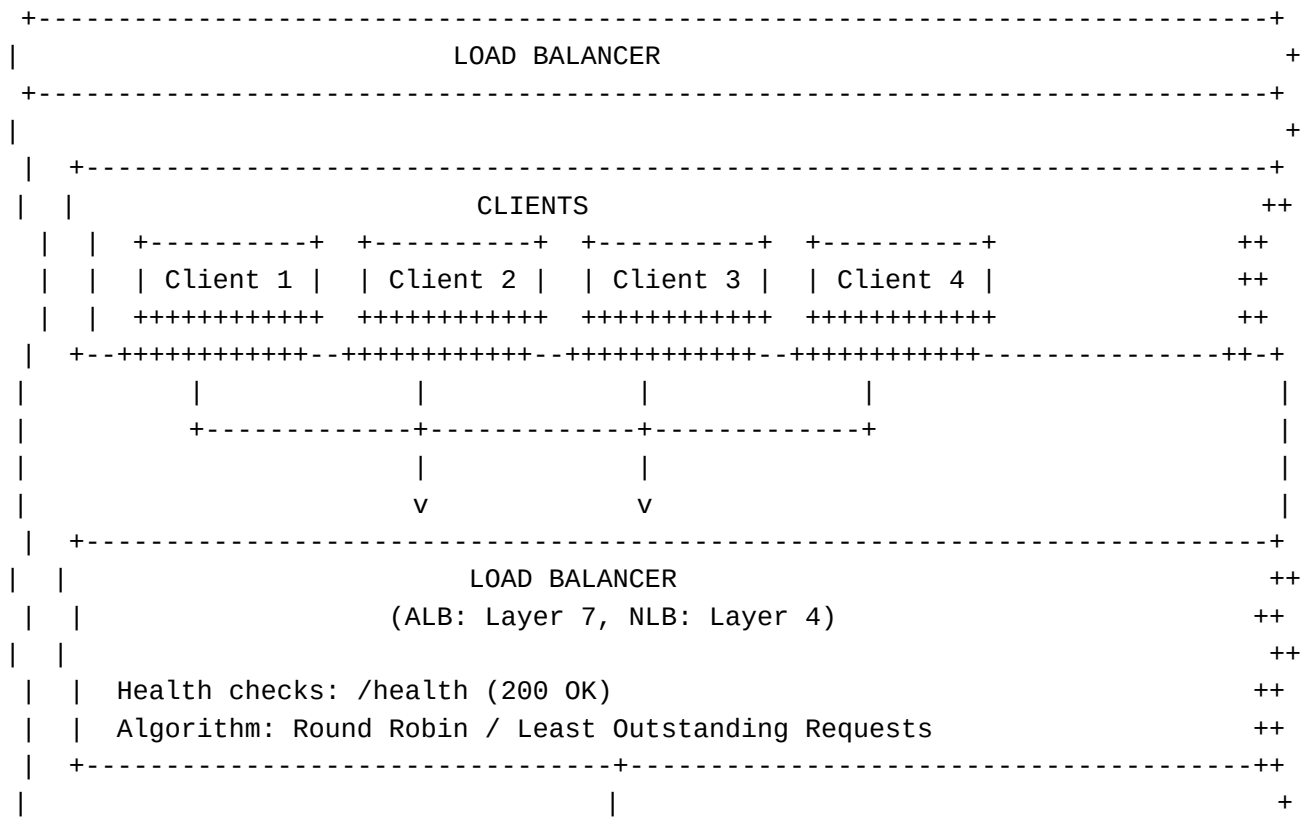
1. **Isolation:** Your own private network in AWS
2. **Control:** Define IP ranges, routing, security
3. **Scalability:** Add subnets, gateways as needed
4. **Hybrid connectivity:** VPN or Direct Connect to on-premises

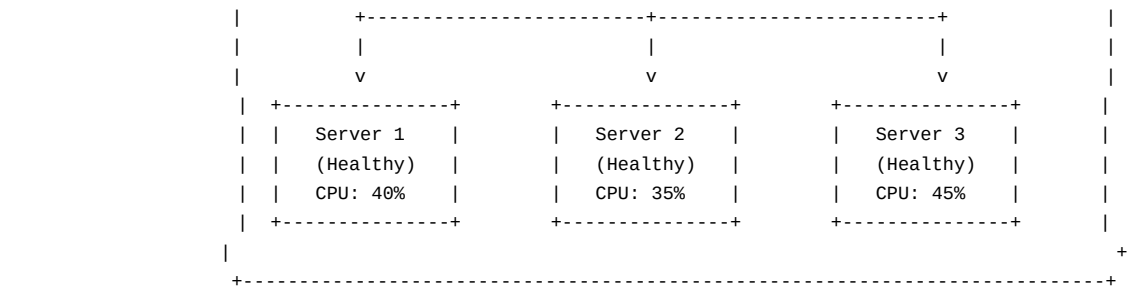
Disadvantages of VPC

1. **Complexity:** Need networking knowledge
2. **Planning required:** CIDR choices can't be changed easily
3. **Limits:** Default limits (5 VPCs per region, 200 subnets per VPC)

Chapter 9: Load Balancers (ALB vs NLB)

What is a Load Balancer?
Distributes incoming traffic across multiple targets (EC2 instances, Lambda functions, IP addresses).
Real Scenario
Your web application runs on 3 EC2 instances. You need to distribute user traffic across them.
Without Cloud (Alternative):
- Set up HAProxy or Nginx as load balancer
- Configure health checks manually
- Handle failover
- Manage scaling
With AWS Load Balancer:
- Create ALB in minutes
- Automatic health checks
- Integration with Auto Scaling
- Fully managed
Diagram: Load Balancer





ALB (Application Load Balancer) - Layer 7

Feature
Description

Protocols
HTTP, HTTPS, gRPC
Routing
Path-based (/api/*, /admin/*), host-based (api.example.com)
Targets
EC2, Lambda, ECS, IP addresses
Features
Sticky sessions, authentication, WAF integration

NLB (Network Load Balancer) - Layer 4

Feature
Description

Protocols
TCP, UDP, TLS
Routing
IP-based, port-based
Targets
EC2, IP addresses
Features
Static IP, very low latency, millions of requests/sec

ALB vs NLB Comparison

Aspect
ALB
NLB

Layer
Layer 7 (HTTP/HTTPS)
Layer 4 (TCP/UDP)
Latency
Moderate (HTTP processing)
Very low (pass-through)
Routing
Path, host, query string
IP, port
Static IP
No (use Global Accelerator)
Yes
Targets
EC2, Lambda, ECS, IP
EC2, IP
Use case
Web apps, APIs, microservices
High performance, TCP/UDP
Price
Moderate
Lower

When to Use ALB

Use Case	Why ALB
----------	---------
Web applications	HTTP/HTTPS traffic
Microservices	Path-based routing (/users, /orders)
API Gateway alternative	Simple API needs
Lambda targets	Serverless backends
A/B testing	Weighted target groups

When to Use NLB

Use Case	Why NLB
----------	---------
TCP/UDP applications	Gaming, media streaming
Low latency	Ultra-low latency requirements
Static IP	Whitelisting, on-premises communication
High throughput	Millions of requests per second
TLS termination	Can terminate TLS

Load Balancer Target Group Features

Feature	Description
---------	----------
Health checks	Regular checks to identify healthy targets
Deregistration delay	Wait for in-flight requests to complete
Sticky sessions	Route same user to same target (ALB only)
Weighted routing	Route different percentages to different targets

---

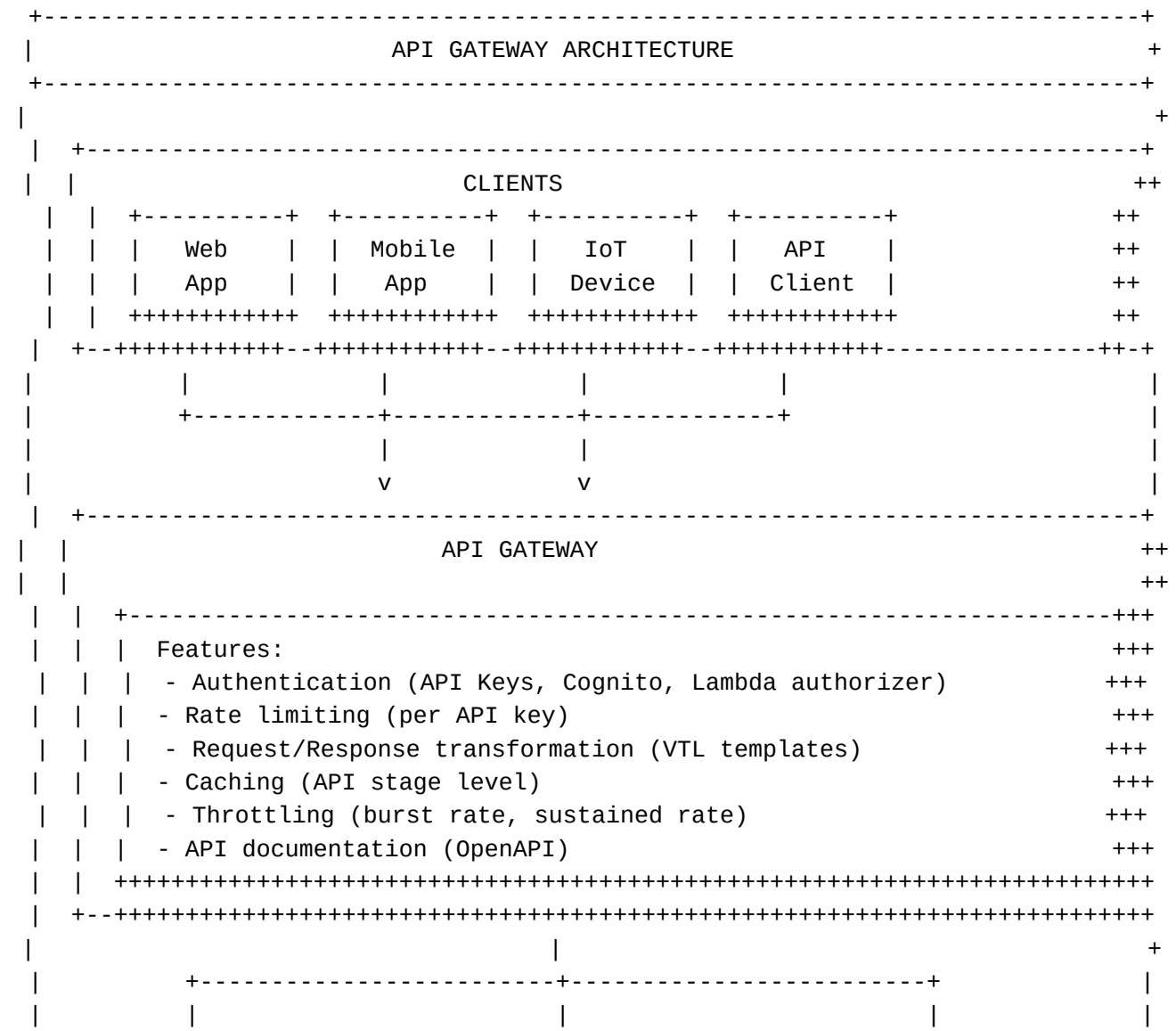
[Page 12]

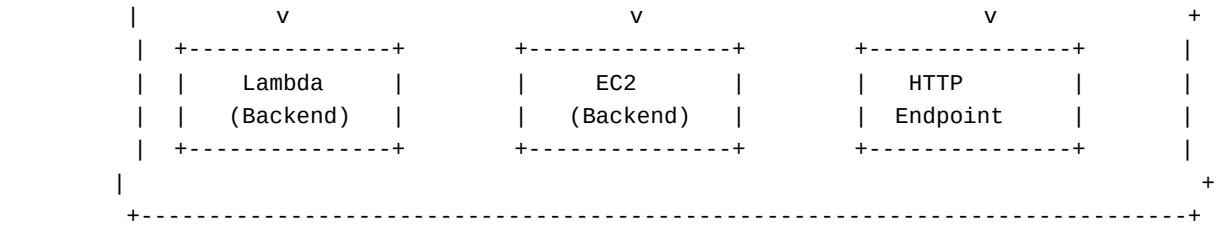
Chapter 10: API Gateway

What is API Gateway?

Fully managed service to create, publish, maintain, monitor, and secure APIs. Acts as a front door for backend services.

Real Scenario
You have microservices (Lambda, EC2) and need a single entry point for your frontend application.
Without Cloud (Alternative):
- Run API server (Express.js, Flask)
- Implement authentication
- Set up rate limiting
- Handle request/response transformation
With API Gateway:
- Define API endpoints in minutes
- Built-in authentication (API keys, Cognito, Lambda authorizers)
- Built-in rate limiting and throttling
- Request/response transformation
Diagram: API Gateway Architecture





API Gateway Example (OpenAPI/Swagger)

```
openapi: 3.0.0
info:
  title: My API
  version: 1.0.0
paths:
  /users:
    get:
      summary: Get all users
      responses:
        '200':
          description: Successful response
      x-amazon-apigateway-integration:
        type: AWS_PROXY
        httpMethod: POST
        uri: arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:123456789012:function:GetUsers/invocations
  /users/{id}:
    get:
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: string
      x-amazon-apigateway-integration:
        type: AWS_PROXY
        httpMethod: POST
        uri: arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:123456789012:function:GetUser/invocations
```

API Gateway Features

Feature	Description
API types	REST, HTTP, WebSocket
Authentication	API keys, Cognito, Lambda authorizers, IAM
Rate limiting	Per API key or per method
Caching	API stage caching (300-3600 seconds)
Transformation	VTL templates for request/response mapping
Monitoring	CloudWatch metrics and logs
Custom domains	Custom domain names with SSL
Usage plans	Throttling and quota per API key

When to Use API Gateway

Use Case	Why API Gateway
----------	----------
Serverless APIs	Frontend for Lambda functions
Microservices	Single entry point for multiple services
Mobile backends	Optimize for mobile clients
Third-party APIs	Monetize API with usage plans
WebSocket APIs	Real-time communication

Advantages of API Gateway

1. **Fully managed:** No servers to manage
2. **Built-in security:** Authentication, rate limiting, throttling
3. **Cost-effective:** Pay per request (no idle cost)
4. **Integration:** Works with Lambda, EC2, HTTP endpoints
5. **Observability:** CloudWatch metrics and logs

Disadvantages of API Gateway

1. **Cold starts:** For Lambda integrations (in milliseconds)
2. **Latency:** Adds ~10-50ms overhead
3. **Complexity:** VTL templates can be complex
4. **Limits:** 10,000 requests per second (soft limit)
5. **Cost:** Can be expensive at high volume

[Page 13]

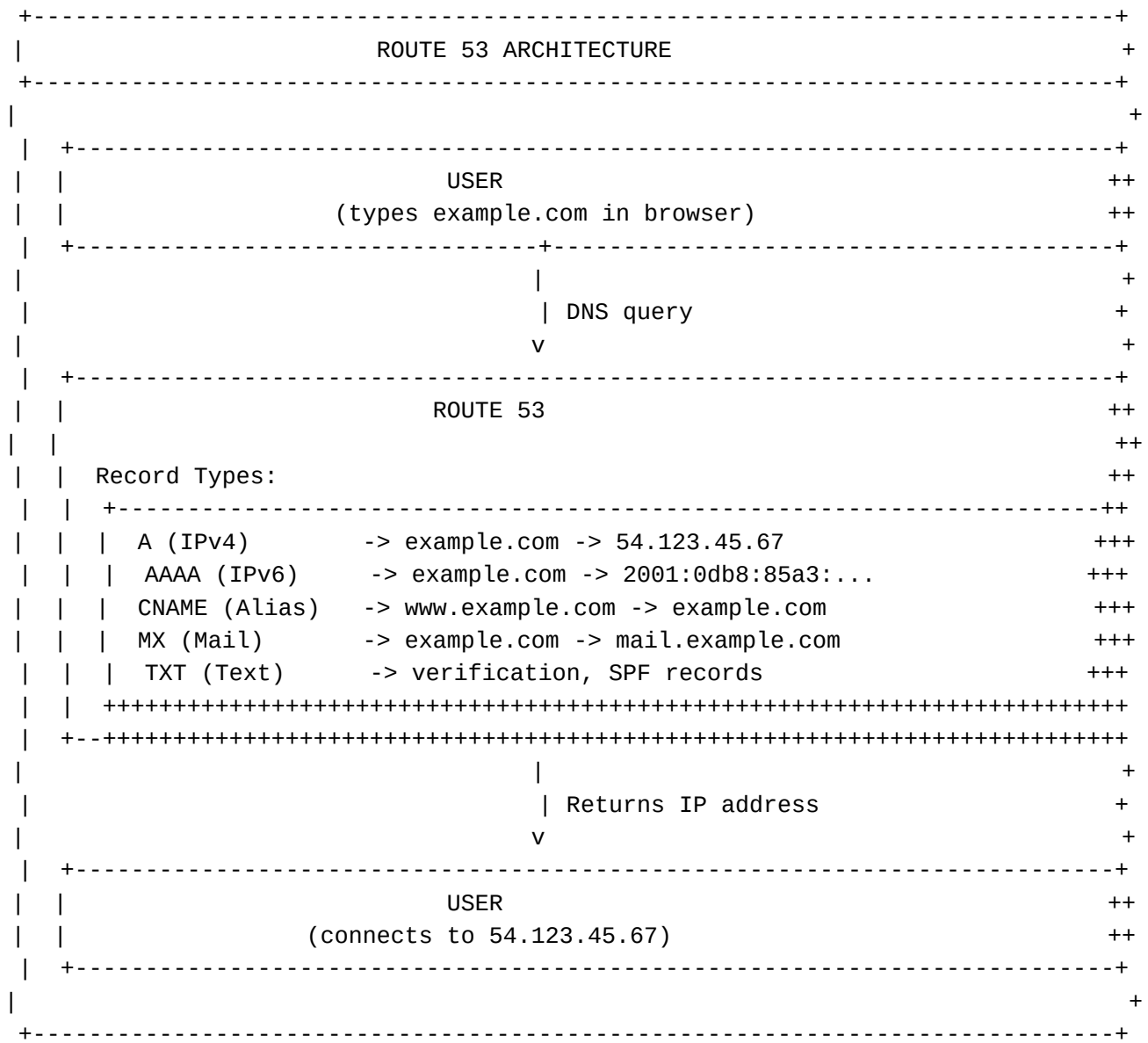
Chapter 11: Route 53 (DNS)

What is Route 53?

AWS's Domain Name System (DNS) service. Translates domain names (example.com) to IP addresses (54.123.45.67).

Real Scenario

You own example.com and want users to reach your website hosted on AWS.
Without Cloud (Alternative):
- Buy domain from registrar
- Set up DNS with provider
- Limited routing options
- Manual health checks
With Route 53:
- Register domain (or transfer)
- Create hosted zone
- Advanced routing (latency, failover, weighted)
- Built-in health checks
Diagram: Route 53 Routing



Route 53 Routing Policies

Policy	Description	Use Case
--------	-------------	----------
Simple	Single resource (one record)	Basic DNS
Weighted	Distribute percentage across resources	A/B testing, blue-green
Latency	Route to lowest latency region	Global applications
Failover	Primary + secondary with health checks	Disaster recovery
Geolocation	Route based on user location	Region-specific content
Multivalue	Up to 8 healthy records	Simple load balancing

Route 53 Health Checks

Feature	Description
---------	-------------
Endpoint monitoring	HTTP, HTTPS, TCP checks
Interval	10 seconds (fast) or 30 seconds (standard)
Failure threshold	1-10 consecutive failures
String matching	Check response content
CloudWatch alarm	Use existing alarms

When to Use Route 53

Use Case	Why Route 53
----------	--------------
Domain registration	Buy or transfer domains
DNS hosting	Create hosted zones
Global load balancing	Latency-based routing
Disaster recovery	Failover routing
A/B testing	Weighted routing

Advantages of Route 53

1. **Global:** DNS queries answered from edge locations
2. **Fast:** Low-latency DNS resolution
3. **Highly available:** 100% SLA
4. **Advanced routing:** Latency, failover, weighted, geolocation
5. **Integrated:** Works with AWS health checks

Disadvantages of Route 53

1. **Cost:** Charges per query (millions of queries cost money)
2. **Propagation:** DNS changes take time (TTL)
3. **Complexity:** Advanced policies need understanding

[Page 14]

PART FIVE: DATABASE SERVICES

Chapter 12: RDS (Relational Database Service)

What is RDS?

Managed SQL databases (PostgreSQL, MySQL, MariaDB, Oracle, SQL Server). AWS handles backups, patching, failover, and scaling.

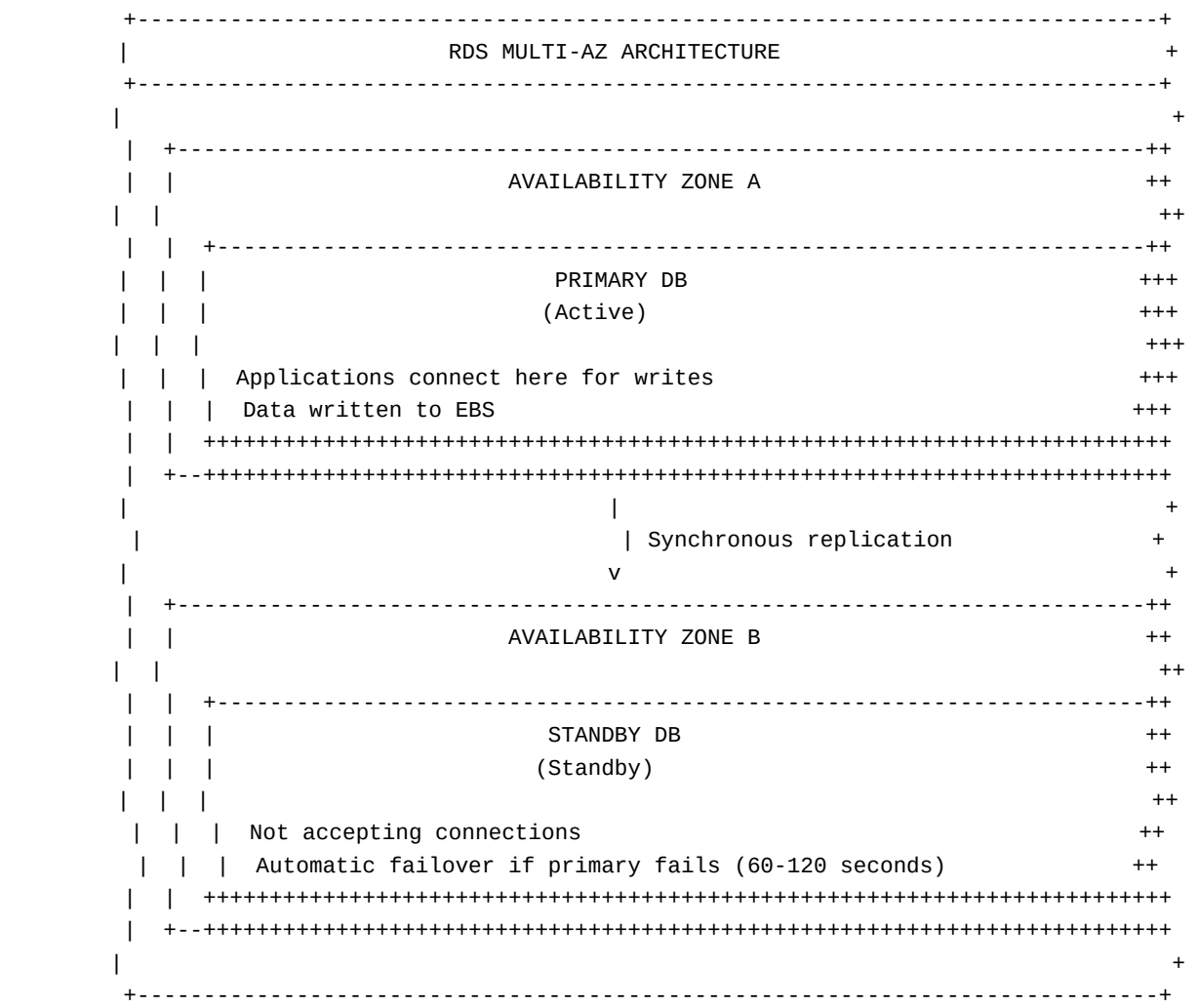
Real Scenario

Your e-commerce application needs a transactional database for orders and customers.

Without Cloud (Alternative):
- Set up database server
- Install and configure database software
- Manage backups (schedule, test restore)
- Handle failover (manual)
- Apply security patches

With RDS:
- Launch in minutes
- Automated backups (point-in-time recovery)
- Multi-AZ for automatic failover
- Automated patching

- Read replicas for scaling
Diagram: RDS Multi-AZ Architecture



RDS Features
Feature
Description

Multi-AZ
Synchronous replication to standby (failover)
Read Replicas
Asynchronous replication for read scaling (up to 15)
Automated backups
Point-in-time recovery (1-35 days)
Manual snapshots
User-initiated backups (keep indefinitely)
Performance Insights
Database performance monitoring
Encryption
At rest (KMS) and in transit (SSL)
Auto scaling
Automatically scale storage
RDS Instance Types
Family
Use Case

Family	Use Case
Standard (m5, m6g)	General purpose
Memory Optimized (r5, x2g)	High memory (caching, analytics)
Burstable (t3, t4g)	Dev/test, light workloads

When to Use RDS

Use Case	Why RDS
----------	---------
Transactional applications	ACID compliance
Complex queries	JOINs, aggregations
Existing SQL apps	MySQL, PostgreSQL, etc.
Reporting	Analytics with read replicas

Advantages of RDS

1. **Managed:** Backups, patching, failover handled
2. **Multi-AZ:** Automatic failover (99.99% availability)
3. **Read scaling:** Up to 15 read replicas
4. **Security:** Encryption, VPC isolation, IAM
5. **Monitoring:** Performance Insights, CloudWatch

Disadvantages of RDS

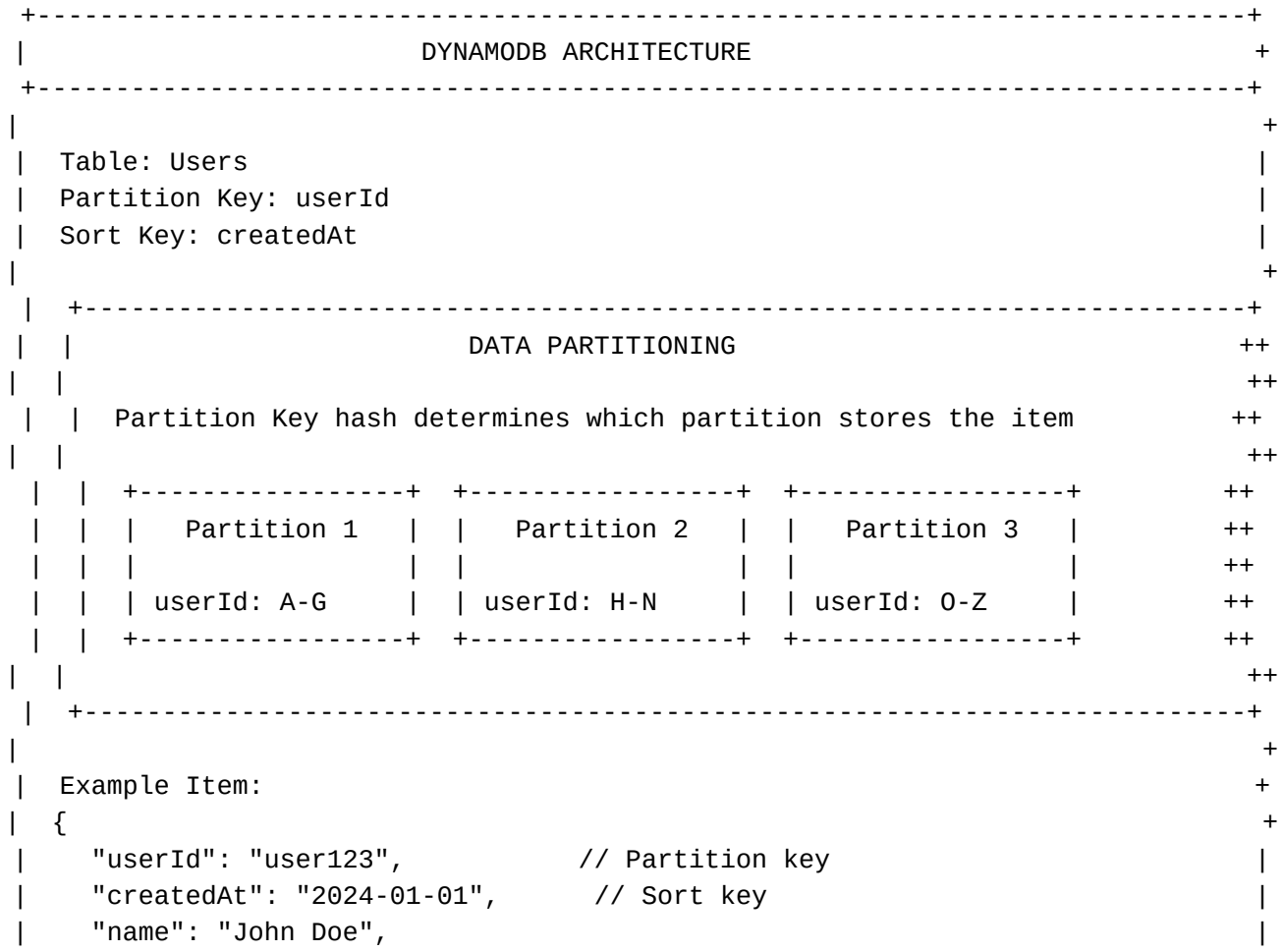
1. **Cost:** More expensive than self-managed
2. **Limited control:** No OS access
3. **Scaling:** Vertical only (bigger instance)
4. **Downtime:** Some changes require downtime

[Page 15]

Chapter 13: DynamoDB (NoSQL Database)

What is DynamoDB?

Fully managed NoSQL database with single-digit millisecond latency at any scale.
Key-value and document data model.
Real Scenario
Your gaming application needs to store player scores with high write throughput (millions of writes per second).
Without Cloud (Alternative):
- Set up NoSQL cluster (Cassandra, MongoDB)
- Configure sharding, replication
- Handle scaling and node failures
- Manage backups and monitoring
With DynamoDB:
- Create table in minutes
- Automatic scaling based on throughput
- Single-digit millisecond latency
- Fully managed (no servers)
Diagram: DynamoDB Architecture



```
|    "email": "john@example.com",                                                |
|    "score": 1500,                                                              |
|    "preferences": {                                                            |
|      "theme": "dark",                                                          |
|      "notifications": true                                                     |
|    }                                                                           |
|  }                                                                             |
|                                                                                +
 +-------------------------------------------------------------------------------+
```

DynamoDB Features

Feature

Description

Capacity modes

On-demand (auto scaling) or Provisioned (manual)

Global tables

Multi-region, active-active replication

DynamoDB Streams

Change data capture (CDC)

TTL (Time to Live)

Auto-delete expired items

Backups

On-demand and continuous (PITR)

Transactions

ACID across multiple items (limited)

DAX

In-memory cache for DynamoDB

Read/Write Capacity Modes

Mode

Description

Use Case

On-demand

Auto-scales based on traffic

Unpredictable workloads

Provisioned

Manual capacity (RCU/WCU)

Predictable workloads

Auto Scaling

Automatic within limits

Mixed workloads

Access Patterns

Pattern

DynamoDB Support

Get by primary key

```
+ Excellent
```

Get by secondary index

```
+ Good
```

Range queries

```
+ Sort key
```

Full table scan

```
++ Expensive
```

Complex joins

```
+ Not supported
```

Aggregations (SUM, AVG)

```
+ Not supported
```

When to Use DynamoDB

Use Case	Why DynamoDB
----------	----------
High throughput	Millions of requests/sec
Low latency	Single-digit milliseconds
Key-value lookups	Get by primary key
Session storage	High-speed access
Leaderboards	With sort key
Event sourcing	With DynamoDB Streams

When NOT to Use DynamoDB

Use Case	Better Option
----------	----------
Complex queries	RDS (SQL)
Joins	RDS
Aggregations	RDS or Analytics
Transactions across many items	RDS
Variable access patterns	RDS or Elasticsearch

Advantages of DynamoDB

1. **Serverless:** No servers to manage
2. **Automatic scaling:** On-demand capacity
3. **Low latency:** Single-digit milliseconds
4. **High throughput:** Millions of requests/sec
5. **Multi-region:** Global tables
6. **Durability:** 11 nines (99.999999999%)

Disadvantages of DynamoDB

1. **Limited querying:** No JOINs, aggregations
2. **Cost:** Can be expensive at high throughput
3. **Flexibility:** Schema flexibility = you manage access patterns
4. **Transactions:** Limited ACID support

RDS vs DynamoDB (Preview)

Aspect	RDS	DynamoDB
--------	-----	----------
Type	SQL (relational)	NoSQL
Scaling	Vertical (bigger instance)	Horizontal (automatic)
Queries	Complex (JOINs, aggregations)	Simple (key lookup)
Latency	Millisecond	Single-digit millisecond
Cost	Predictable (instance-based)	Usage-based (RCU/WCU)
Best for	Complex queries, transactions	High scale, simple lookups

(Detailed comparison in Chapter 25)

[Page 16]

PART SIX: DEVOPS & CI/CD

Chapter 14: CodePipeline, CodeBuild, CodeDeploy

What is CodePipeline?

Fully managed continuous delivery service that automates build, test, and deploy phases.

What is CodeBuild?

Fully managed build service that compiles source code, runs tests, and produces artifacts.

What is CodeDeploy?

Fully managed deployment service that automates deployments to EC2, ECS, Lambda, and on-premises.

Real Scenario

You want to automatically deploy code changes from GitHub to production.

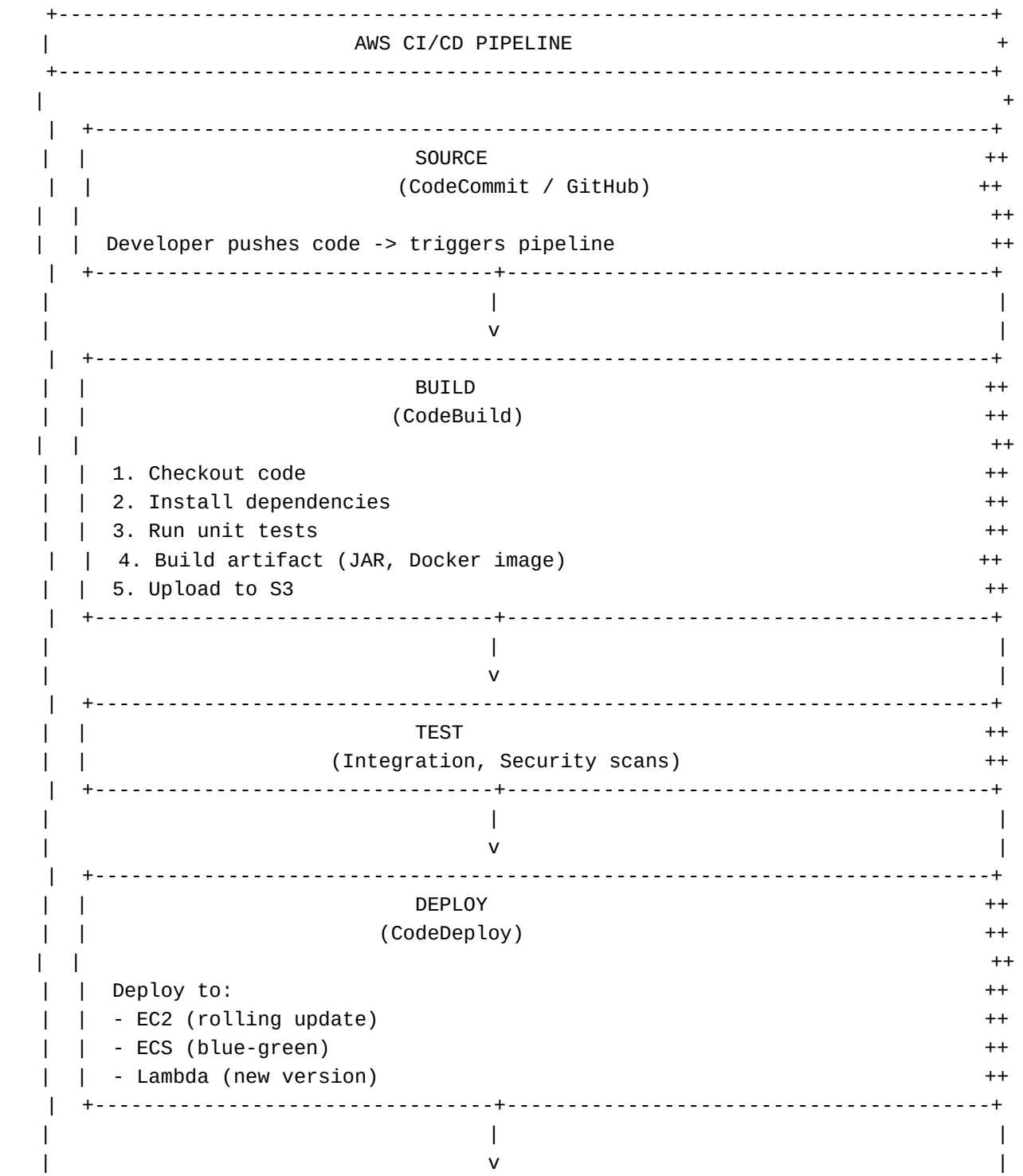
Without Cloud (Alternative):

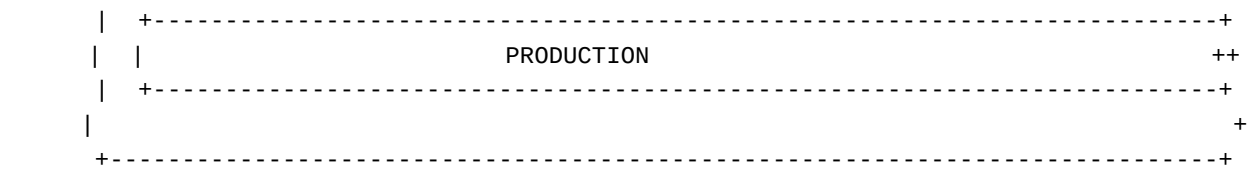
- Set up Jenkins server
- Configure build agents
- Manage plugins
- Handle scaling and maintenance

With AWS CI/CD:

- CodePipeline triggers on git push
- CodeBuild runs tests
- CodeDeploy deploys to EC2
- Fully managed, no servers

Diagram: AWS CI/CD Pipeline





CodeBuild Example (buildspec.yml)

```
version: 0.2
phases:
install:
                                        runtime-versions:
nodejs: 18
commands:
                                           - npm install
pre_build:
commands:
                                           - npm run lint
                                           - npm run test
build:
commands:
                                          - npm run build
post_build:
commands:
                                    - echo Build completed on 'date'
artifacts:
files:
                                              - '**/*'
base-directory: dist
cache:
paths:
                                         - node_modules/**/*
```

CodeDeploy Deployment Strategies

Strategy	Description	Use Case
In-place	Update existing instances	Simple, lower cost
Blue-green	Create new instances, shift traffic	Zero downtime, rollback
Canary	Gradual traffic shift	Risk reduction
Linear	Incremental traffic shift	Controlled rollout

When to Use AWS CI/CD

Use Case	Why AWS CI/CD
AWS-centric	Deep integration with AWS services

Use Case	Why AWS CI/CD
Managed service	No Jenkins servers to manage
Simple to set up	Minutes to create pipeline
Security	IAM integration

Advantages of AWS CI/CD

1. **Fully managed:** No servers to manage
2. **Integration:** Works with GitHub, Bitbucket, S3
3. **Pay per use:** No idle cost
4. **Security:** IAM roles for permissions
5. **Scalability:** Automatic scaling

Disadvantages of AWS CI/CD

1. **Vendor lock-in:** AWS-specific
2. **Complex features:** Advanced pipelines need custom scripts
3. **UI:** Can be clunky for complex workflows

[Page 17]

Chapter 15: CloudFormation & CDK (IaC)

What is CloudFormation?

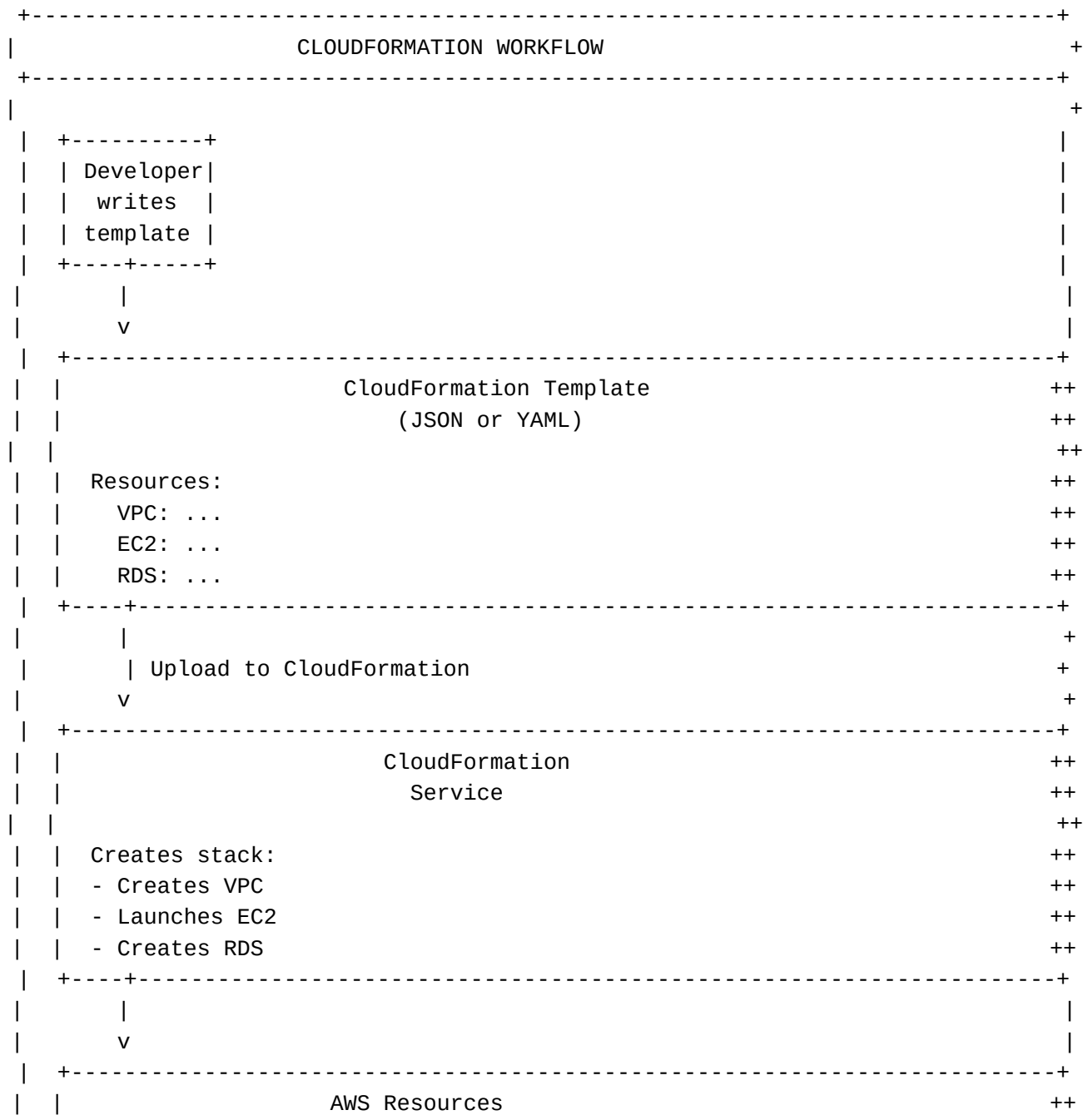
Infrastructure as Code (IaC) service that models and provisions AWS resources using templates (JSON/YAML).

What is CDK?

AWS Cloud Development Kit (CDK) lets you define cloud infrastructure using familiar programming languages (TypeScript, Python, Java, C#).

Real Scenario

You need to create the same environment (VPC, EC2, RDS) for dev, test, and production.
Without Cloud (Alternative):
- Manually click in AWS console (error-prone)
- Write scripts (boto3, AWS CLI)
- No version control
- Inconsistent environments
With CloudFormation/CDK:
- Define infrastructure as code
- Version control (Git)
- Repeatable deployments
- Consistent environments
Diagram: CloudFormation Workflow



```
 |  |              (VPC, EC2, RDS, etc.)                                          ++
  |  +-------------------------------------------------------------------------+
|                                                                                 +
 +---------------------------------------------------------------------------+
```

CloudFormation Template Example (YAML)

```
AWSTemplateFormatVersion: '2010-09-09'
Description: 'My application stack'
Parameters:
Environment:
Type: String
Default: dev
AllowedValues:
                                        - dev
                                       - prod
Resources:
# VPC
MyVPC:
Type: AWS::EC2::VPC
Properties:
CidrBlock: 10.0.0.0/16
EnableDnsSupport: true
EnableDnsHostnames: true
Tags:
                                  - Key: Name
                             Value: !Sub '${Environment}-vpc'
# EC2 Instance
MyEC2Instance:
Type: AWS::EC2::Instance
Properties:
InstanceType: t3.micro
                           ImageId: ami-0c55b159cbfafe1f0
SubnetId: !Ref PublicSubnet
Tags:
                                  - Key: Name
                        Value: !Sub '${Environment}-web-server'
# RDS Database
MyDatabase:
Type: AWS::RDS::DBInstance
Properties:
DBInstanceClass: db.t3.micro
Engine: postgres
EngineVersion: '13'
MasterUsername: !Ref DBUsername
MasterUserPassword: !Ref DBPassword
AllocatedStorage: 20
StorageType: gp2
Outputs:
WebsiteURL:
Value: !Sub 'http://${MyEC2Instance.PublicDnsName}'
Description: URL for the website
```

CDK Example (TypeScript)

```
import * as cdk from 'aws-cdk-lib';
```

```
import * as ec2 from 'aws-cdk-lib/aws-ec2';
import * as rds from 'aws-cdk-lib/aws-rds';

export class MyStack extends cdk.Stack {
  constructor(scope: cdk.App, id: string, props?: cdk.StackProps) {
    super(scope, id, props);

    // Create VPC
    const vpc = new ec2.Vpc(this, 'MyVpc', {
      maxAzs: 2,
      natGateways: 1,
    });

    // Create EC2 instance
    const webServer = new ec2.Instance(this, 'WebServer', {
      vpc,
      instanceType: ec2.InstanceType.of(
        ec2.InstanceClass.T3,
        ec2.InstanceSize.MICRO
      ),
      machineImage: ec2.MachineImage.latestAmazonLinux2(),
    });

    // Create RDS database
    const database = new rds.DatabaseInstance(this, 'Database', {
      engine: rds.DatabaseInstanceEngine.postgres({
        version: rds.PostgresEngineVersion.VER_13,
      }),
      instanceType: ec2.InstanceType.of(
        ec2.InstanceClass.T3,
        ec2.InstanceSize.MICRO
      ),
      vpc,
    });
  }
}
```

When to Use CloudFormation/CDK

Use Case	Why CloudFormation/CDK
----------	----------
Infrastructure as Code	Version-controlled infrastructure
Repeatable deployments	Dev, test, prod consistency
Disaster recovery	Rebuild infrastructure quickly
Team collaboration	Code reviews, Git history

Advantages of CloudFormation

1. **Declarative:** Define desired state, AWS handles differences
2. **Rollback:** Automatic rollback on failure
3. **Dependency management:** Automatic resource ordering
4. **Drift detection:** Detect manual changes

5. **Integration:** Works with all AWS services

Disadvantages of CloudFormation

1. **Complex syntax:** JSON/YAML can be verbose
2. **Slow:** Updates can be slow (10-30 minutes)
3. **Limited logic:** No loops, conditionals limited
4. **State management:** Stack state can get stuck

CloudFormation vs CDK

Aspect	CloudFormation	CDK
--------	----------	-----
Language	JSON/YAML	TypeScript, Python, Java, C#
Abstraction	Low-level	High-level (constructs)
Learning curve	Moderate	Lower for developers
Reusability	Limited	Composable constructs
Best for	Ops teams	Developers

[Page 18]

Chapter 16: CloudWatch (Monitoring & Logging)

What is CloudWatch?

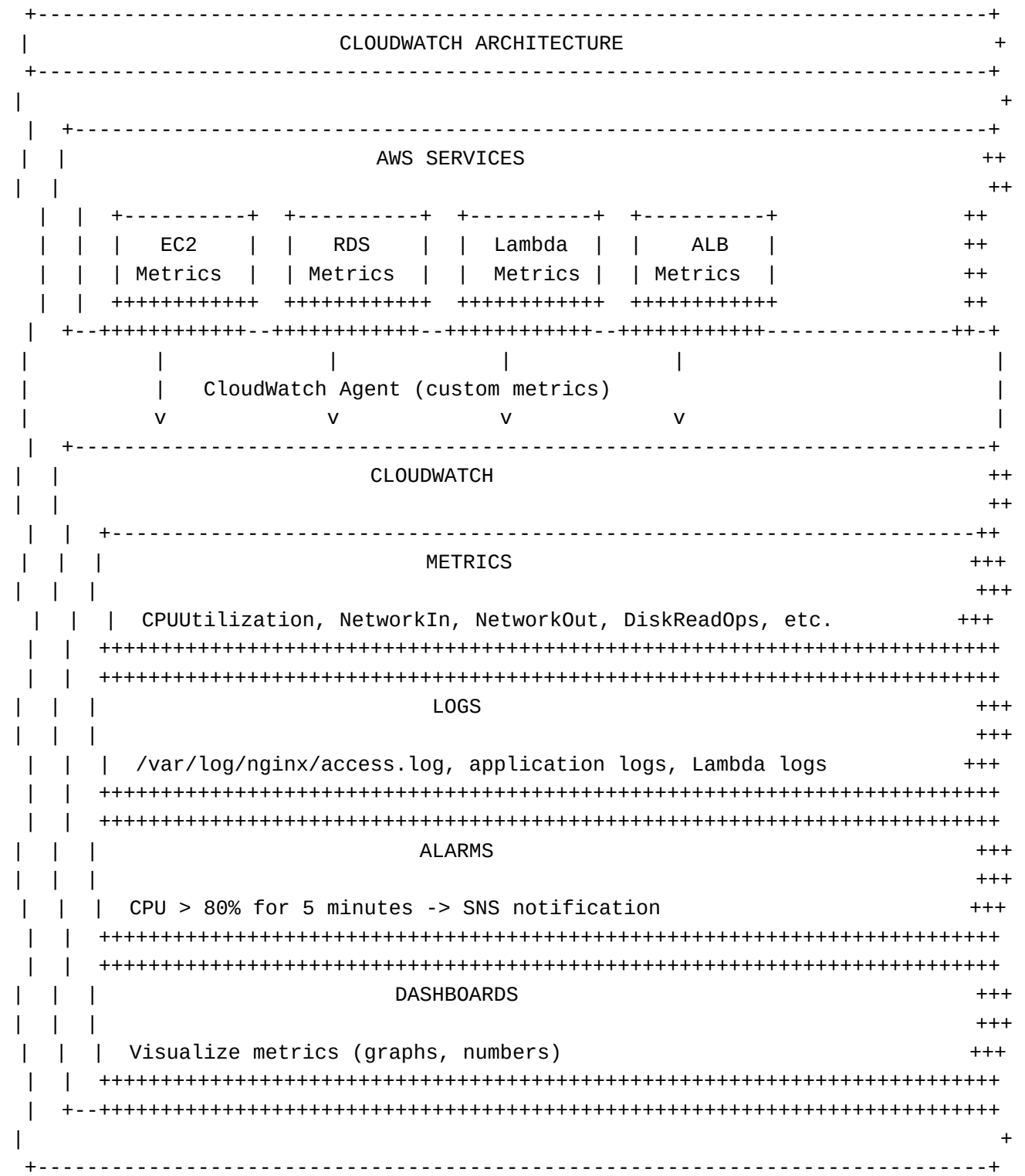
Monitoring and observability service for AWS resources and applications.

Real Scenario

You need to monitor your EC2 instance CPU usage and get alerted when it exceeds 80%.

Without Cloud (Alternative):
- Set up monitoring server (Nagios, Zabbix)
- Configure agents on each server
- Manage alerting system

- Handle scaling
With CloudWatch:
- Automatic metrics for EC2, RDS, Lambda
- Create alarm in minutes
- SNS notification on trigger
- Fully managed
Diagram: CloudWatch Architecture



CloudWatch Metrics

Service	Common Metrics
---	---
EC2	CPUUtilization, NetworkIn, NetworkOut, DiskReadOps, DiskWriteOps
RDS	CPUUtilization, DatabaseConnections, FreeStorageSpace, ReadLatency
Lambda	Invocations, Duration, Errors, Throttles
ALB	RequestCount, TargetResponseTime, HTTPCode_ELB_5XX
S3	NumberOfObjects, BucketSizeBytes, AllRequests

CloudWatch Logs Features

Feature	Description
---	---
Log groups	Container for log streams
Log streams	Sequence of log events
Log retention	Keep logs for 1 day to 10 years
Metric filters	Extract metrics from logs
Subscription filters	Stream logs to Lambda, Kinesis
Logs Insights	Query logs with SQL-like syntax

CloudWatch Alarms

Component	Description
---	---
Metric	What to monitor (CPUUtilization)
Period	How often to evaluate (5 minutes)
Threshold	When to trigger (> 80)
Evaluation	How many periods (3 consecutive)
Action	What to do (SNS, Auto Scaling)

CloudWatch Logs Insights Query Example

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```

When to Use CloudWatch

Use Case	Why CloudWatch
---	---

Use Case	Why CloudWatch
Monitoring AWS services	Built-in metrics
Application logging	Centralize logs
Alerts	Set up notifications
Dashboards	Visualize metrics
Auto Scaling	Use metrics for scaling policies

Advantages of CloudWatch

1. **Integrated:** Works with all AWS services
2. **Managed:** No servers to manage
3. **Scalable:** Handles millions of metrics
4. **Cost-effective:** Pay per metric (first 10 free)

Disadvantages of CloudWatch

1. **Cost:** Can be expensive at scale (high-resolution metrics)
2. **Retention:** Limited default retention
3. **Query language:** Not as powerful as Splunk or ELK
4. **Lag:** Metrics can take minutes to appear

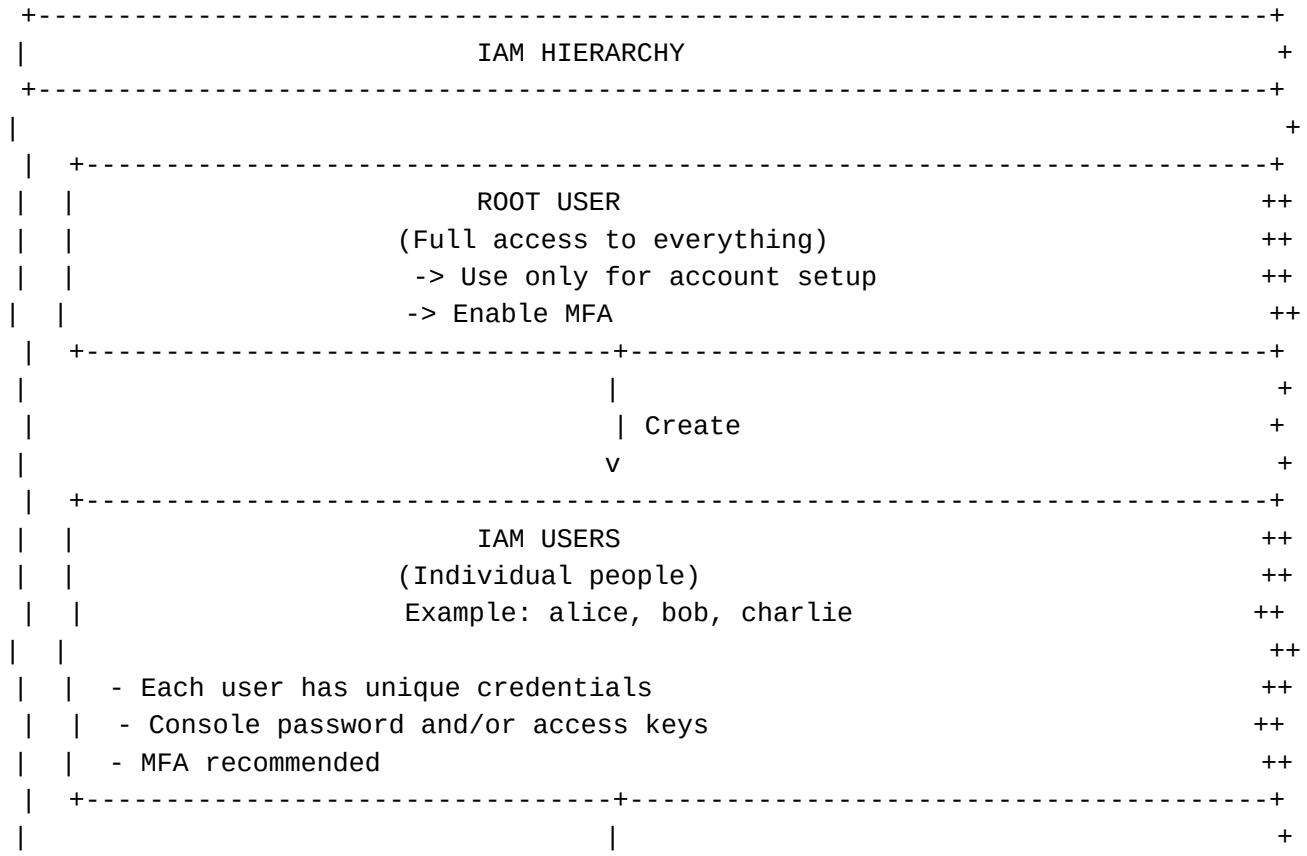
CloudWatch vs Third-Party Tools

Tool	Use Case
---	---
CloudWatch	AWS-native monitoring
Prometheus	Open source metrics
Datadog	Comprehensive SaaS monitoring
ELK Stack	Log aggregation and analysis
Grafana	Visualization (can use CloudWatch as data source)

[Page 19]

PART SEVEN: SECURITY

Chapter 17: IAM (Identity and Access Management)
What is IAM?
Manage access to AWS services and resources. Create users, groups, roles, and define permissions with policies.
Real Scenario
Your development team needs access to EC2 (create, stop, start), but not to billing or IAM.
Without Cloud (Alternative):
- Separate AWS accounts (hard to manage)
- Shared root credentials (dangerous)
- No fine-grained permissions
With IAM:
- Create IAM users for each developer
- Assign to "Developers" group
- Attach policy allowing EC2 actions
- Enforce MFA for privileged actions
Diagram: IAM Hierarchy





IAM Policy Structure

```
{
"Version": "2012-10-17",
"Statement": [
{
"Sid": "AllowEC2Actions",
"Effect": "Allow",
"Action": [
"ec2:DescribeInstances",
"ec2:StartInstances",
"ec2:StopInstances"
],
"Resource": "*"
},
{
"Sid": "DenyBillingAccess",
"Effect": "Deny",
                                    "Action": "aws-portal:*",
"Resource": "*"
}
]
}
```

IAM Best Practices

Practice	Why
----------	-----
No root user access	Root has full permissions, use only for initial setup
Enable MFA	Adds security layer for privileged actions
Use groups	Manage permissions by role, not per user
Least privilege	Give minimum permissions needed
Use roles for EC2	Never store credentials on EC2
Rotate keys regularly	Reduce risk of compromised keys
Audit policies	Use IAM Access Analyzer, CloudTrail
Use conditions	Restrict by IP, time, MFA

When to Use IAM

Use Case	Why IAM
----------	---------
User management	Create AWS users for employees
Service permissions	Give EC2 access to S3 via roles
Federated access	Connect corporate Active Directory
Fine-grained control	Restrict by resource, action, condition

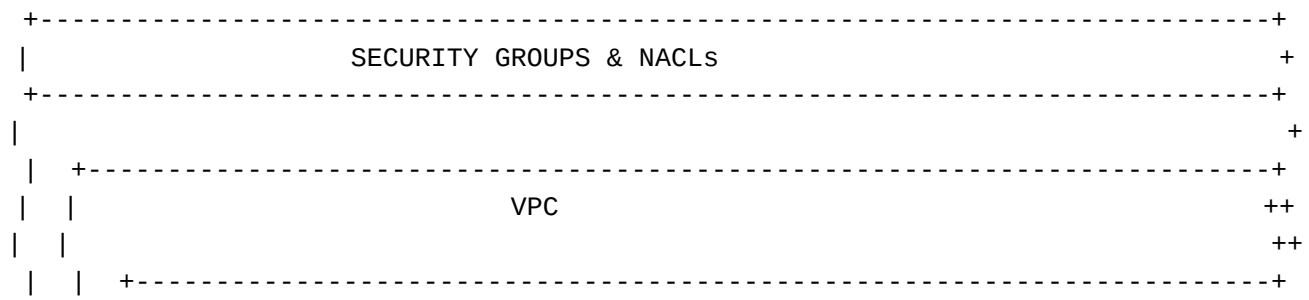
Advantages of IAM

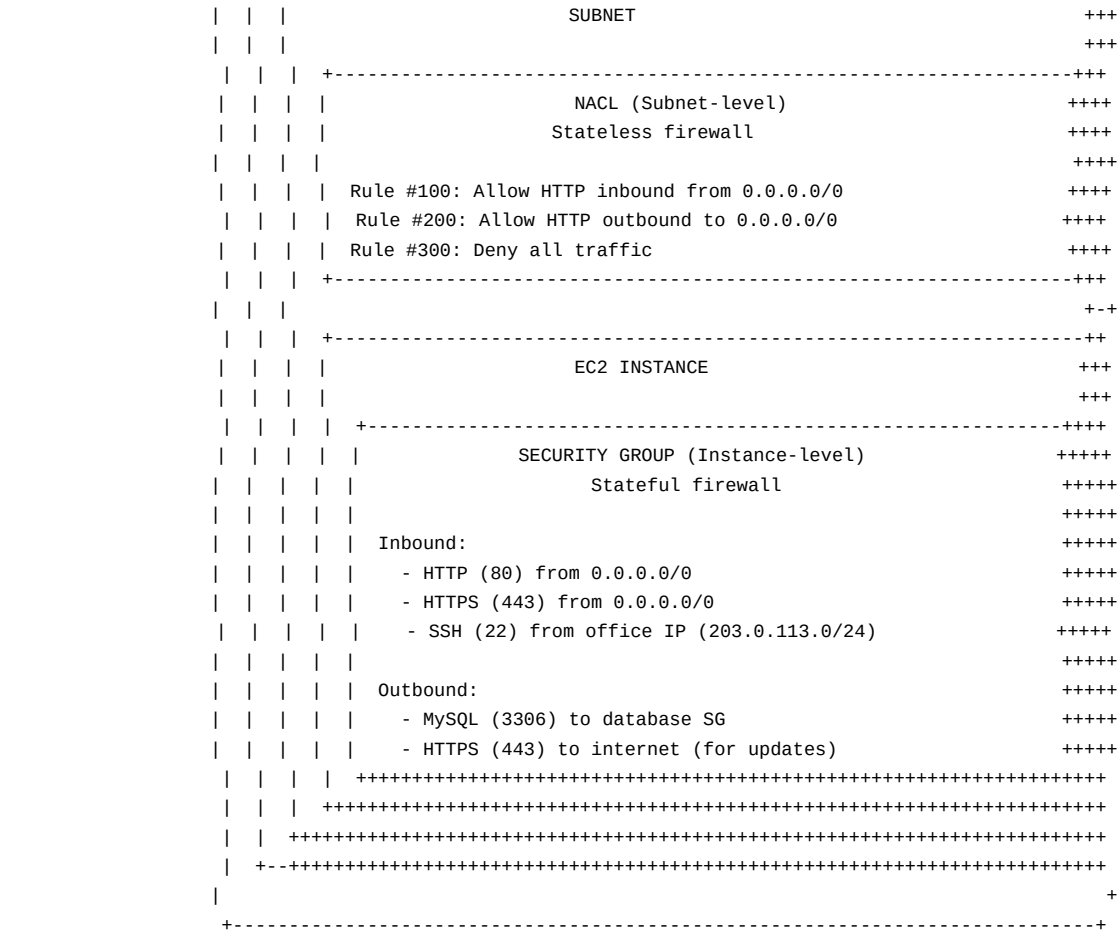
1. **Fine-grained control:** Allow/deny specific actions on specific resources
2. **Centralized management:** One place for all permissions
3. **Integration:** Works with all AWS services
4. **Federation:** Connect to Active Directory, Google, etc.
5. **Audit:** CloudTrail logs all IAM actions

Disadvantages of IAM

1. **Complexity:** Policies can become complex
2. **Propagation delays:** Changes take seconds to minutes
3. **Quotas:** Limits on users, groups, roles (can be increased)
4. **Global:** IAM is global (affects all regions)

[Page 20]
Chapter 18: Security Groups & NACLs
What are Security Groups?
Virtual firewall for EC2 instances. Control inbound and outbound traffic at the instance level.
What are NACLs?
Network Access Control Lists. Stateless firewall for subnets.
Real Scenario
Your web server should accept HTTP/HTTPS from anywhere, but SSH only from office IP. Your database should accept only from web server.
Without Cloud (Alternative):
- Configure iptables on each server
- Manual updates
- No centralized management
With Security Groups:
- Create security group for web servers
- Allow HTTP (0.0.0.0/0), SSH (office IP)
- Create security group for database
- Allow MySQL from web security group
Diagram: Security Groups & NACLs





Security Groups vs NACLs

Aspect	Security Group	NACL
Level	Instance	Subnet
State	Stateful (return traffic allowed)	Stateless (rules both ways)
Rules	Only Allow	Allow and Deny
Order	All rules evaluated	Rules in numeric order (first match)
Default	Deny all inbound, allow all outbound	Allow all inbound and outbound
Use case	Instance-level security	Subnet-level defense in depth

Common Security Group Rules

Rule	Type	Protocol	Port	Source	Purpose
SSH	Inbound	TCP	22		

Rule	Type	Protocol	Port	Source	Purpose
MySQL	Inbound	TCP	3306	App SG	Database access
Outbound	Outbound	All	All	0.0.0.0/0	General internet

Best Practices

Practice	Why
----------	-----
Use security groups	Simpler, stateful
Least privilege	Only open required ports
Restrict SSH	Only from office IP or bastion
Use security group references	Reference by ID, not IP
Audit rules	Regular review of open ports
Log VPC Flow Logs	Monitor traffic patterns

[Page 21]

Chapter 19: Secrets Manager & KMS

What is Secrets Manager?

Securely store, rotate, and manage secrets (database credentials, API keys, OAuth tokens).

What is KMS (Key Management Service)?

Create and manage encryption keys. Used by many AWS services for encryption at rest.

Real Scenario

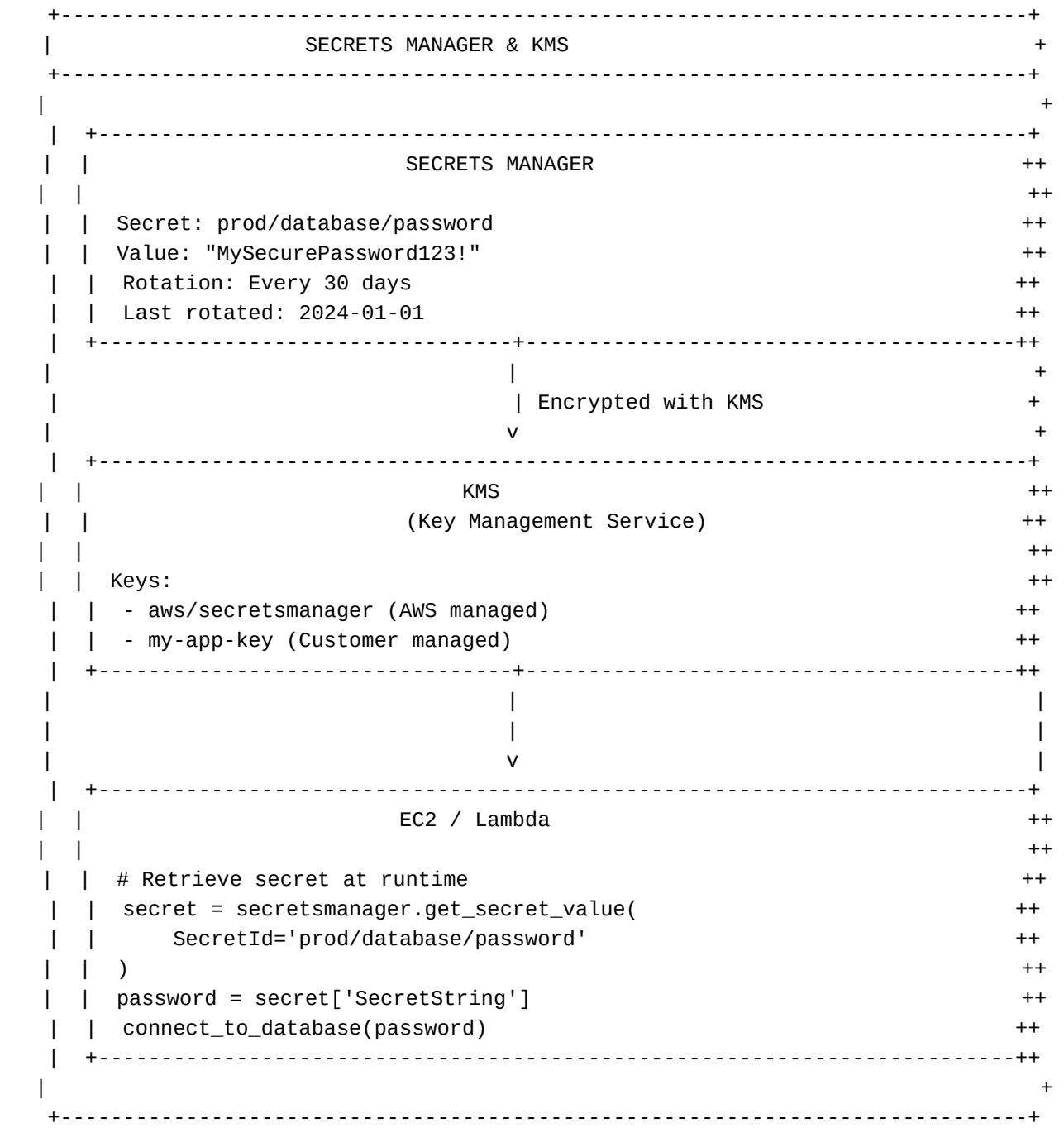
Your application needs database credentials. You don't want to hardcode them in code or config files.

Without Cloud (Alternative):

- Store in config files (security risk)

- Use environment variables (still visible)

- Manual rotation (error-prone)
With Secrets Manager:
- Store secret in Secrets Manager
- Retrieve via API at runtime
- Automatic rotation (30 days)
- Audit access with CloudTrail
Diagram: Secrets Manager & KMS



Secrets Manager vs Parameter Store
Aspect
Secrets Manager
Parameter Store

Aspect	Secrets Manager	Parameter Store
Cost	$0.40 per secret/month	$0.05 per parameter/month
Rotation	Built-in automatic rotation	Manual or Lambda
Encryption	Always encrypted	Optional (SecureString)
Secret size	Up to 10 KB	Up to 8 KB
Use case	Secrets (passwords, API keys)	Config values (database URL)

KMS Key Types

Type	Description	Use Case
---	---	---
AWS managed	Created by AWS (aws/service-name)	Default encryption
Customer managed	You create and manage	Custom requirements
Custom key store	Keys in CloudHSM	Regulatory compliance

When to Use Secrets Manager & KMS

Use Case	Why Secrets Manager	Why KMS
---	---	---
Database passwords	Automatic rotation	Encryption
API keys	Secure storage	Encryption
Third-party tokens	Central management	Encryption
S3 encryption	—	Server-side encryption
EBS encryption	—	Volume encryption

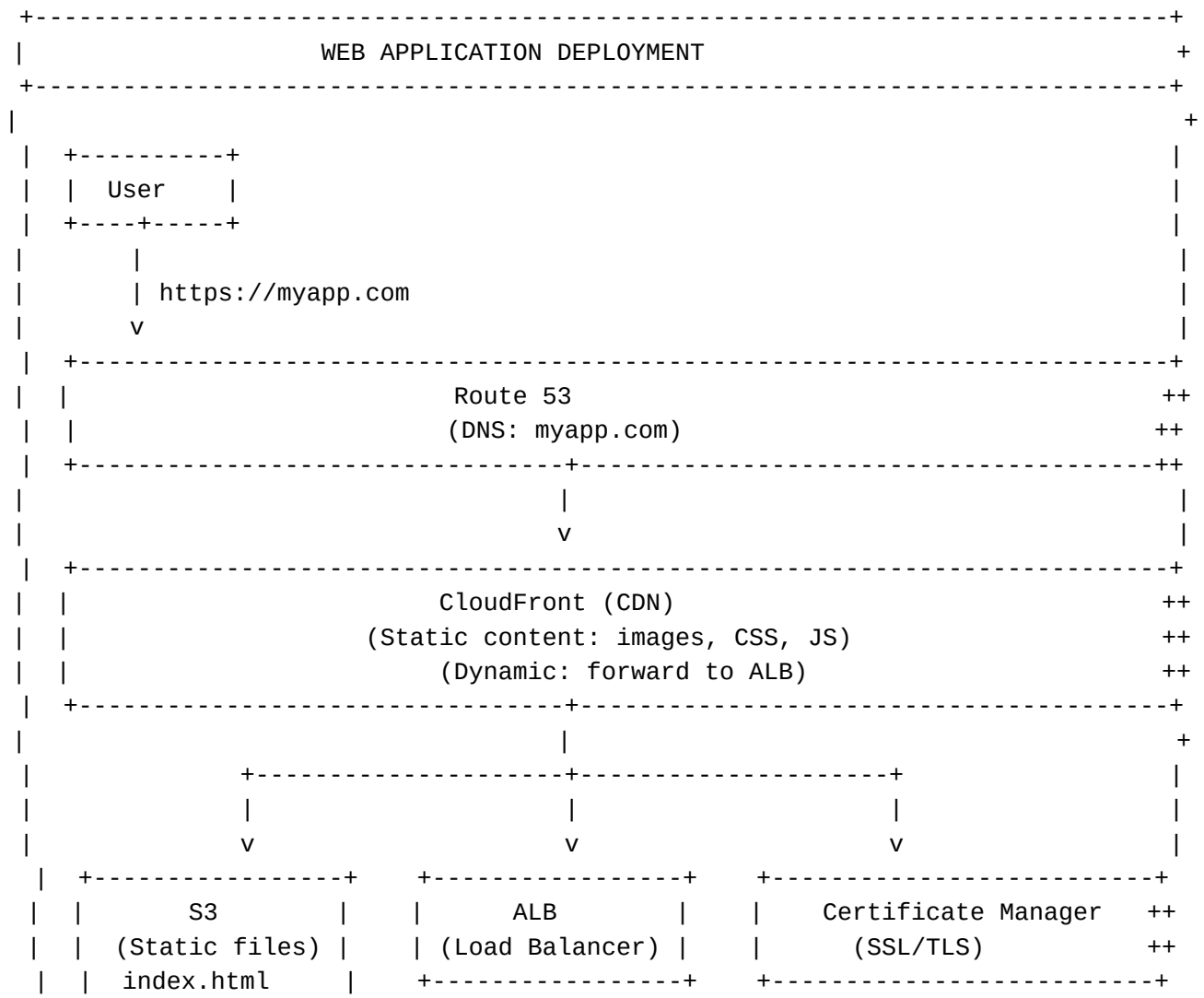
Advantages of Secrets Manager

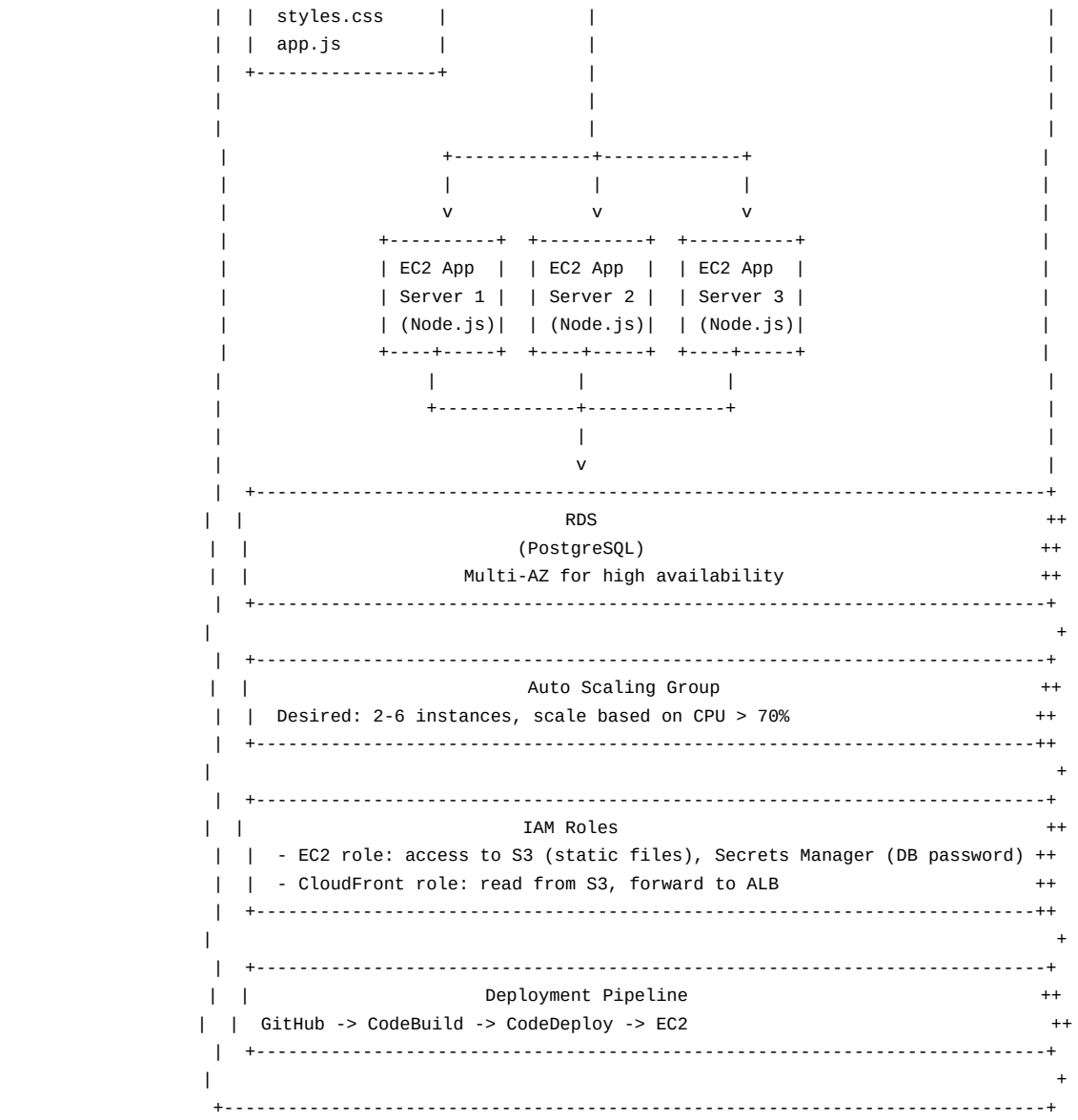
1. **Automatic rotation:** Built-in for RDS, Redshift, DocumentDB
2. **Audit:** CloudTrail logs secret access
3. **Integration:** Works with Lambda for custom rotation
4. **Secure:** Encrypted with KMS

Disadvantages of Secrets Manager

1. **Cost:** More expensive than Parameter Store
2. **Limits:** 500,000 secrets per region
3. **Latency:** API call adds latency

[Page 22]
PART EIGHT: REAL SCENARIOS
Chapter 20: Deploying a Web Application
Scenario
You need to deploy a web application with:
- Static files (HTML, CSS, JS) served from S3
- Backend API running on EC2
- Database for user data (RDS)
- SSL/TLS certificate for HTTPS
- Auto scaling for variable traffic
Complete Architecture Diagram



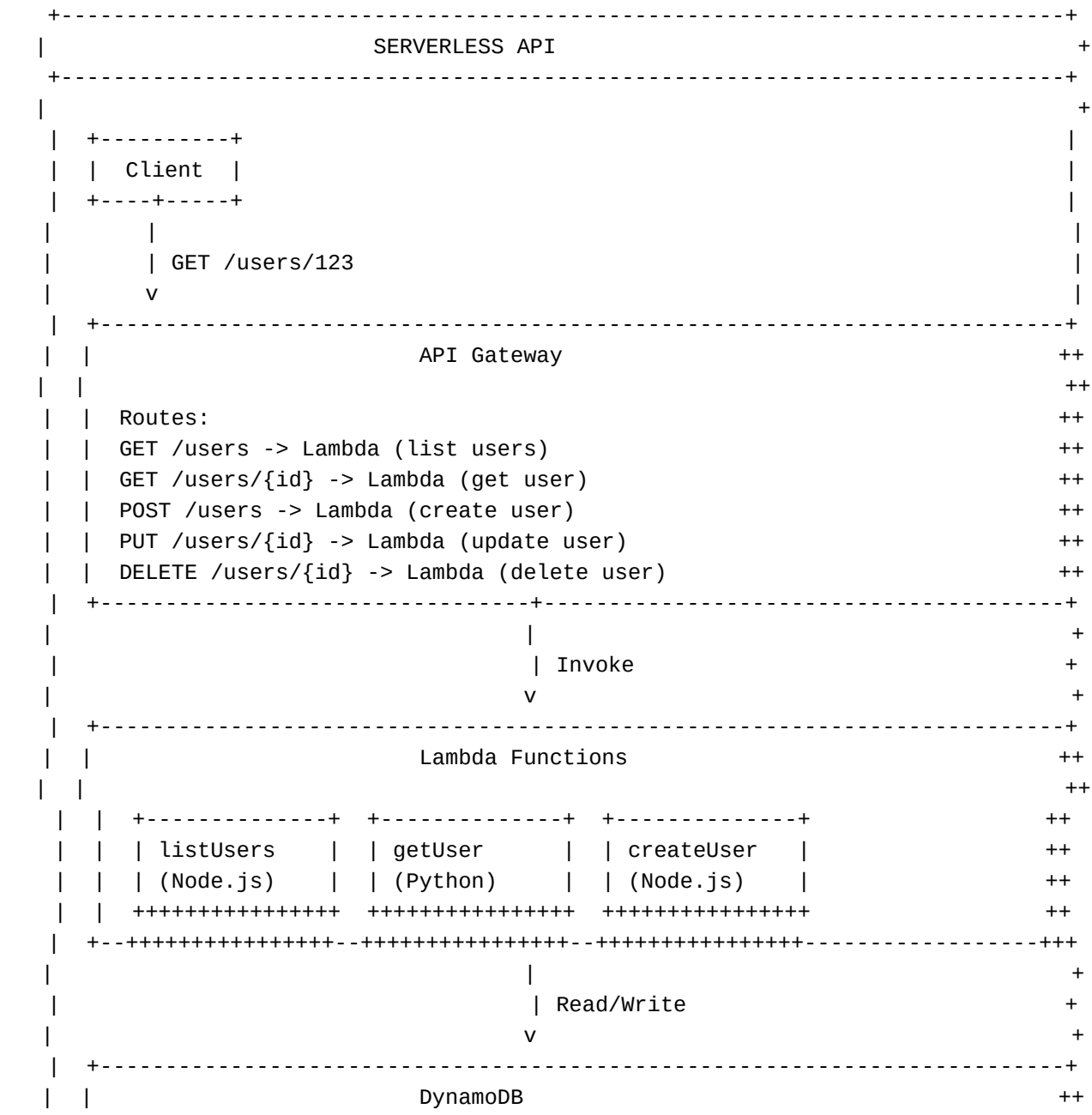


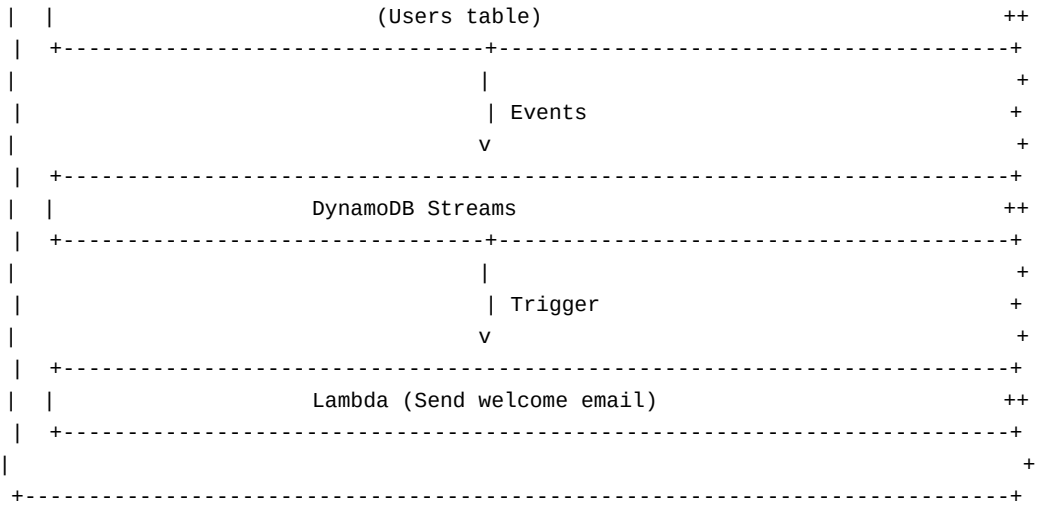
Step-by-Step Deployment

Step	Service	Action
1	Route 53	Register domain, create hosted zone
2	Certificate Manager	Request SSL certificate for myapp.com
3	S3	Upload static files, enable static hosting
4	CloudFront	Create distribution for static files
5	VPC	Create VPC with public and private subnets
6	RDS	Launch PostgreSQL (private subnet)
7	Secrets Manager	Store database password
8	EC2	Launch instances with app code
9	ALB	

Step
Service
Action
10
Auto Scaling
Configure scaling policy
11
CodePipeline
Set up CI/CD from GitHub

[Page 23]
Chapter 21: Building a Serverless API
Scenario
You need a REST API that scales to millions of requests, but you don't want to manage servers.
Architecture





Lambda Function Example (createUser)

```
// createUser Lambda function
const AWS = require('aws-sdk');
const dynamodb = new AWS.DynamoDB.DocumentClient();
const sns = new AWS.SNS();
exports.handler = async (event) => {
try {
// Parse request body
const { name, email } = JSON.parse(event.body);

// Validate input
                                        if (!name || !email) {
return {
statusCode: 400,
body: JSON.stringify({ error: 'Missing name or email' })
};
}

// Generate user ID
                                    const userId = 'user_${Date.now()}';

// Save to DynamoDB
await dynamodb.put({
                                      TableName: process.env.USERS_TABLE,
Item: {
userId,
name,
email,
createdAt: new Date().toISOString()
}
}).promise();

// Publish event to SNS (send welcome email)
await sns.publish({
                                    TopicArn: process.env.WELCOME_TOPIC,
Message: JSON.stringify({ userId, email, name })
}).promise();

return {
statusCode: 201,
                                  headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ userId, name, email })
};
} catch (error) {
```

```
            console.error(error);
            return {
                statusCode: 500,
                body: JSON.stringify({ error: 'Internal server error' })
            };
        }
    };
```

Infrastructure as Code (CDK)

```
    import * as cdk from 'aws-cdk-lib';
    import * as apigateway from 'aws-cdk-lib/aws-apigateway';
    import * as lambda from 'aws-cdk-lib/aws-lambda';
    import * as dynamodb from 'aws-cdk-lib/aws-dynamodb';

    export class ServerlessApiStack extends cdk.Stack {
      constructor(scope: cdk.App, id: string, props?: cdk.StackProps) {
        super(scope, id, props);

        // DynamoDB table
        const usersTable = new dynamodb.Table(this, 'UsersTable', {
          partitionKey: { name: 'userId', type: dynamodb.AttributeType.STRING },
          billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
        });

        // Lambda functions
        const createUserLambda = new lambda.Function(this, 'CreateUserFunction', {
          runtime: lambda.Runtime.NODEJS_18_X,
          handler: 'index.handler',
          code: lambda.Code.fromAsset('lambda/createUser'),
          environment: {
            USERS_TABLE: usersTable.tableName,
          },
        });

        usersTable.grantWriteData(createUserLambda);

        // API Gateway
        const api = new apigateway.RestApi(this, 'UserApi', {
          restApiName: 'User Service',
          description: 'API for user management',
        });

        const users = api.root.addResource('users');
        users.addMethod('POST', new apigateway.LambdaIntegration(createUserLambda));
      }
    }
```

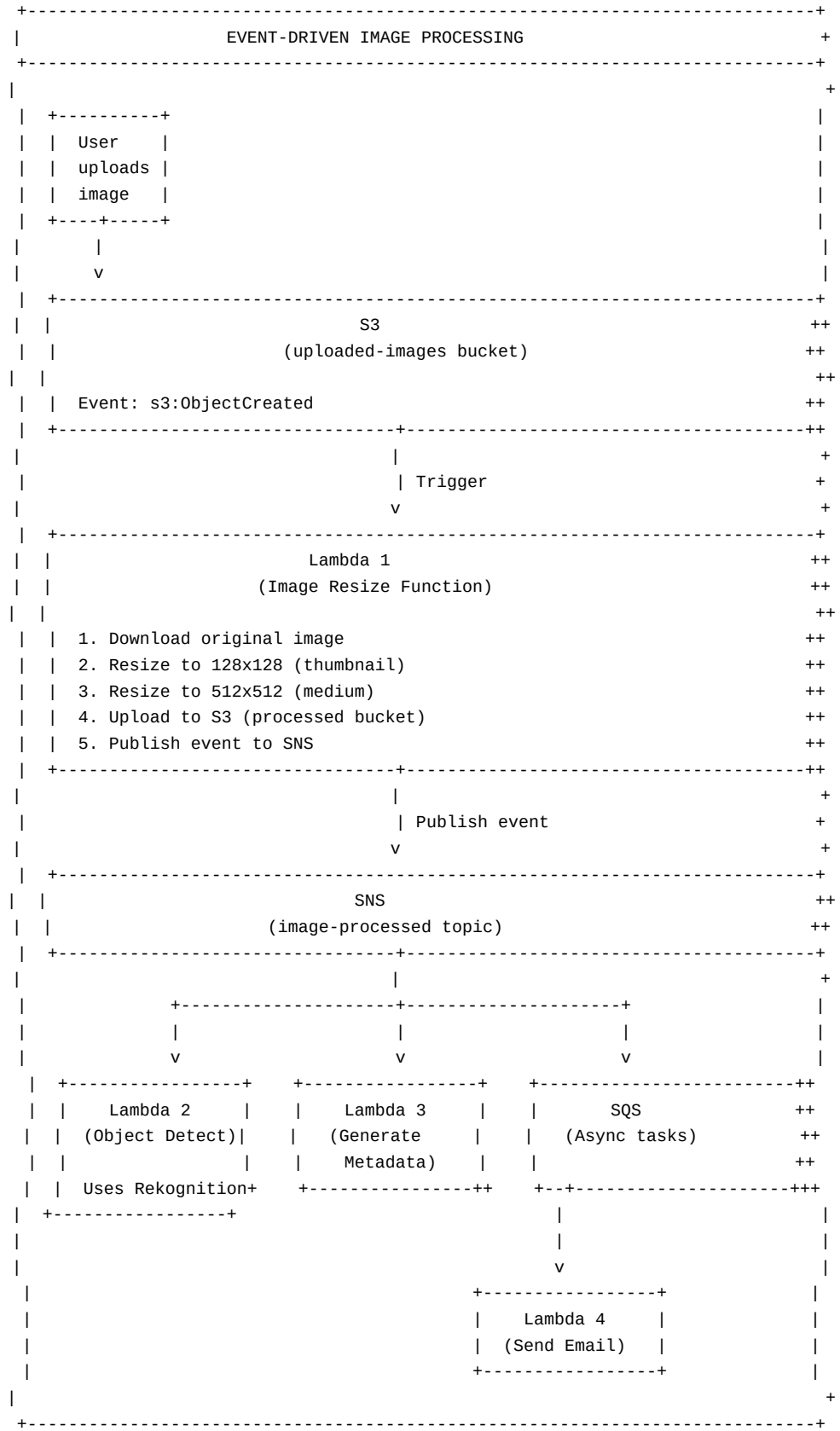
[Page 24]

Chapter 22: Event-Driven Image Processing

Scenario

Users upload images to your application. You need to resize them (multiple sizes), detect objects, and generate thumbnails.

Architecture



Lambda Function (Image Resize)

```
const AWS = require('aws-sdk');
const sharp = require('sharp');
const s3 = new AWS.S3();

exports.handler = async (event) => {
    // Get S3 event
    const bucket = event.Records[0].s3.bucket.name;
    const key = decodeURIComponent(event.Records[0].s3.object.key);

    // Download original image
    const original = await s3.getObject({ Bucket: bucket, Key: key }).promise();
    const image = original.Body;

    // Resize images in parallel
    const [thumbnail, medium] = await Promise.all([
        sharp(image).resize(128, 128).toBuffer(),
        sharp(image).resize(512, 512).toBuffer()
    ]);

    // Upload processed images
    await Promise.all([
        s3.putObject({
            Bucket: process.env.PROCESSED_BUCKET,
            Key: `thumbnail/${key}`,
            Body: thumbnail,
            ContentType: 'image/jpeg'
        }).promise(),
        s3.putObject({
            Bucket: process.env.PROCESSED_BUCKET,
            Key: `medium/${key}`,
            Body: medium,
            ContentType: 'image/jpeg'
        }).promise()
    ]);

    // Notify other services
    const sns = new AWS.SNS();
    await sns.publish({
        TopicArn: process.env.IMAGE_PROCESSED_TOPIC,
        Message: JSON.stringify({ originalKey: key, thumbnailKey: `thumbnail/${key}` })
    }).promise();
};
```

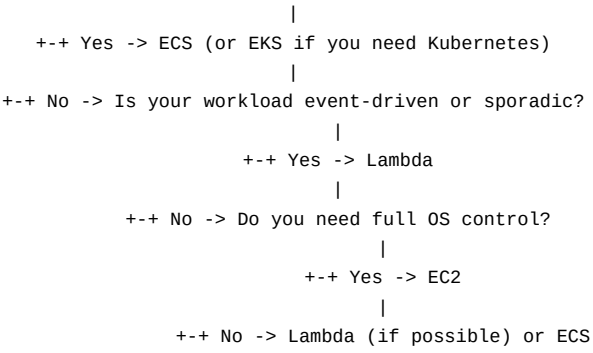
[Page 25]

PART NINE: TOOL COMPARISONS (IMPORTANT)

Chapter 23: EC2 vs Lambda vs ECS

Aspect
EC2
Lambda
ECS

What it is
Virtual server
Serverless function
Container orchestration
Control
Full (OS, software)
None (just code)
Moderate (container)
Pricing
Per hour (even idle)
Per invocation (no idle)
Per hour + resources
Scaling
Manual or Auto Scaling
Automatic
Manual or Auto Scaling
Startup time
Minutes
Milliseconds (cold start)
Seconds
Max duration
Unlimited
15 minutes
Unlimited
State
Can be stateful
Stateless
Can be stateful
Best for
Long-running, custom apps
Event-driven, sporadic
Microservices, portability
Without cloud
Physical server
Always-running server
Kubernetes cluster
When to use
Lift-and-shift, legacy
File processing, APIs
Containers, microservices
Decision Tree
Start: Do you have existing code in containers?



Chapter 24: S3 vs EBS vs EFS
Aspect
S3
EBS
EFS

Type
Object
Block
File
Access

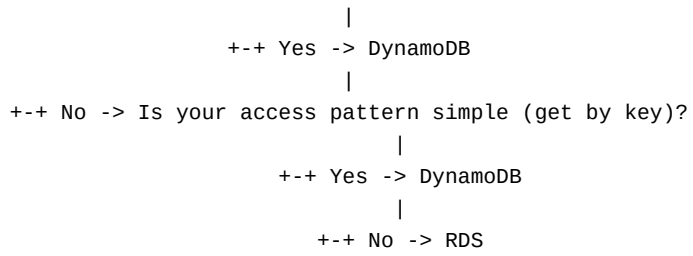
Aspect
S3
EBS
EFS
When to use
Images, videos, static sites
Databases, high I/O
Shared content, multiple servers
Decision Tree
Do you need shared access across multiple EC2 instances?

```
                                                    |
                                                +-+ Yes -> EFS
                                                    |
                                    +-+ No -> Do you need high IOPS/low latency?
                                                        |
                                                    +-+ Yes -> EBS
                                                        |
                                    +-+ No -> Is your data accessed via API (not mounted)?
                                                            |
                                                        +-+ Yes -> S3
                                                            |
                                                +-+ No -> EBS (default choice)
```

Chapter 25: RDS vs DynamoDB
Aspect
RDS
DynamoDB

Type
SQL (relational)
NoSQL (key-value/document)
Schema
Fixed (tables, columns)
Flexible (schema-less)
Scaling
Vertical (bigger instance)
Horizontal (automatic)
Queries
Complex (JOINs, aggregations)
Simple (key lookup)
ACID transactions
Yes (full)
Limited (transactions API)
Latency
Millisecond
Single-digit millisecond
Cost
Predictable (instance-based)
Usage-based (RCU/WCU)
Best for
Complex queries, transactions
High scale, simple lookups
Without cloud
Traditional DB server
NoSQL cluster
When to use
Structured data, SQL needed
High throughput, simple keys
Decision Tree
Do you need complex queries (JOINs, aggregations)?

```
                                                    |
                                                +-+ Yes -> RDS
                                                    |
                                    +-+ No -> Do you need ACID transactions across multiple items?
                                                            |
                                                        +-+ Yes -> RDS
                                                            |
                                        +-+ No -> Do you need high throughput (millions of requests/sec)?
```

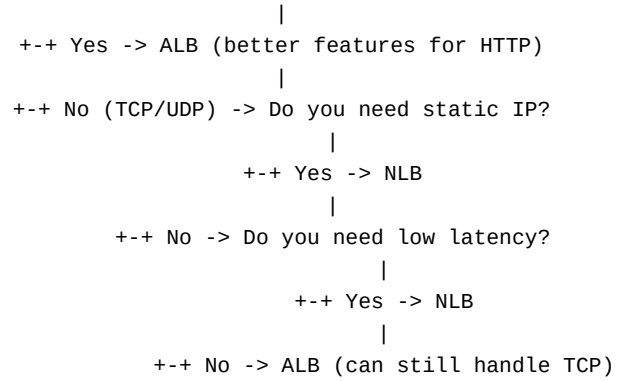


Chapter 26: ALB vs NLB

Aspect	ALB	NLB
Layer	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP)
Routing	Path-based, host-based	IP-based, port-based
Latency	Moderate (HTTP processing)	Very low (pass-through)
Static IP	No (use Global Accelerator)	Yes
Target types	EC2, Lambda, ECS, IP	EC2, IP
Features	Sticky sessions, authentication, WAF	High throughput, low latency
Use case	Web apps, APIs, microservices	Gaming, media streaming, TCP/UDP
Without cloud	HAProxy, Nginx	Hardware load balancer
When to use	HTTP/HTTPS traffic, path routing	TCP/UDP, low latency, static IP

Decision Tree

Is your traffic HTTP/HTTPS?



PART TEN: INTERVIEW PREPARATION

Chapter 27: Common AWS Interview Questions

1. Q: What's the difference between EC2 and Lambda?

A: EC2 is a virtual server you manage (OS, patches, scaling). Lambda is serverless—you just upload code, and AWS runs it on demand. EC2 is for long-running, stateful applications. Lambda is for event-driven, short-running functions. EC2 costs per hour (even idle), Lambda costs per invocation (no idle cost).

2. Q: How do you secure an S3 bucket?

A:

- Block public access by default (block public access settings)
- Use bucket policies for granular access control
- Enable encryption at rest (SSE-S3, SSE-KMS)
- Use IAM roles for application access
- Enable versioning to protect against accidental deletions
- Enable access logging to track requests
- Use pre-signed URLs for temporary access

3. Q: Explain the difference between RDS and DynamoDB.

A: RDS is SQL-based (relational) with fixed schema, supports complex queries (JOINs), and scales vertically (bigger instance). DynamoDB is NoSQL (key-value/document) with flexible schema, scales horizontally automatically, and is best for high throughput with simple lookup patterns. Use RDS for complex transactions and reporting; use DynamoDB for high-scale, low-latency key-value access.

4. Q: What's a VPC and why use private subnets?

A: VPC is a logically isolated network in AWS where you launch resources. Private subnets have no direct internet access, so databases and application servers are protected from external access. Only load balancers and web servers go in public subnets. Private subnets can access internet via NAT Gateway for updates.

5. Q: How does auto scaling work?

A: Auto Scaling Groups maintain desired number of EC2 instances based on policies (CPU > 70% for 5 minutes, scheduled scaling, custom metrics). It launches new instances when needed and terminates unhealthy or idle ones. Works with load balancers to distribute traffic.

6. Q: What is IAM? What's the difference between user, group, role, and policy?

A: IAM manages access to AWS services.

- **User:** Individual person with credentials
- **Group:** Collection of users (same permissions)
- **Role:** Assumed by AWS services (no credentials)
- **Policy:** JSON document defining permissions

7. Q: What's the difference between security group and NACL?

A: Security Group is instance-level, stateful (return traffic automatically allowed), only Allow rules. NACL is subnet-level, stateless (rules needed for both directions), supports Allow and Deny rules with priority order.

8. Q: How would you design a highly available application on AWS?

A:

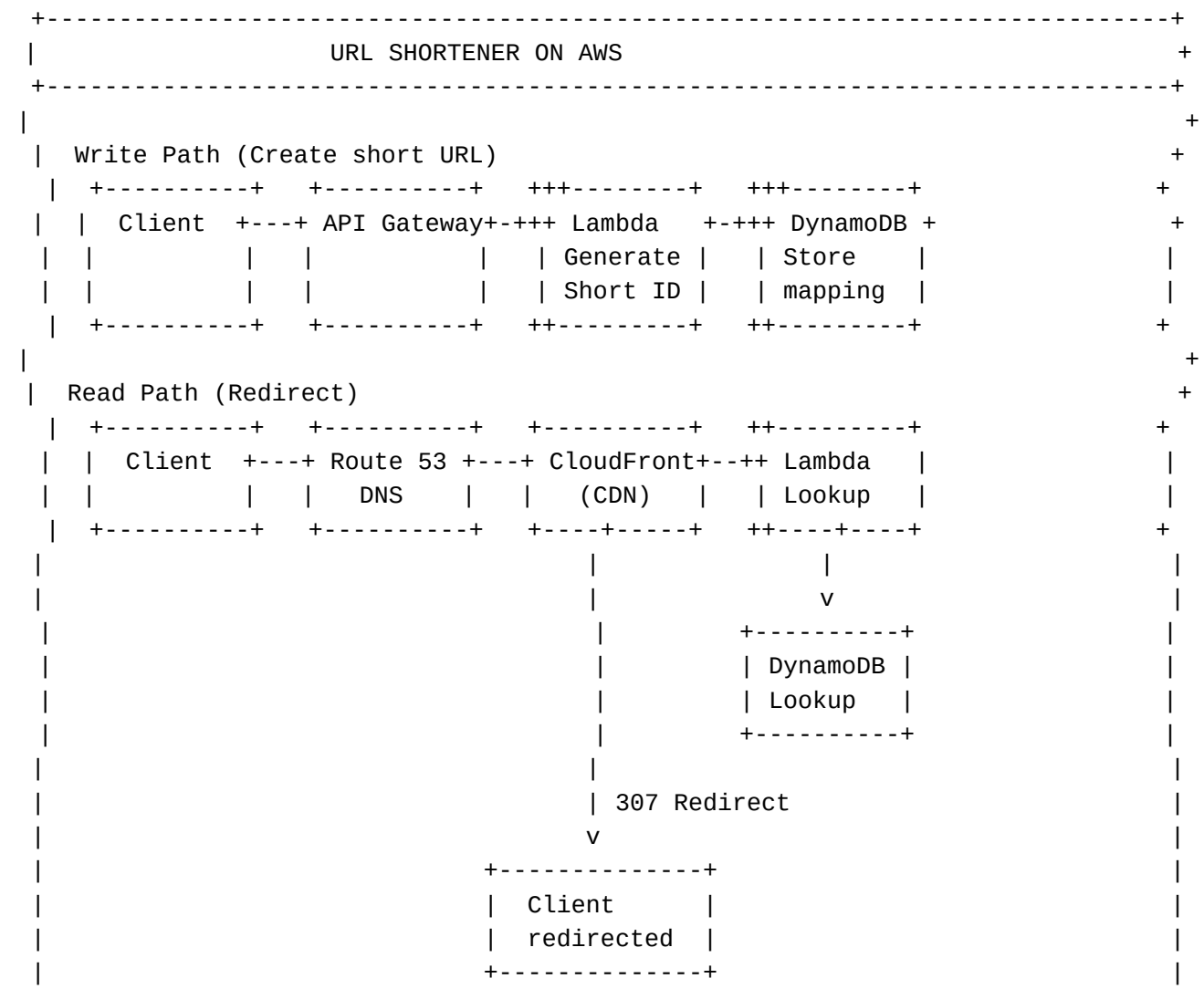
- Deploy across multiple Availability Zones (AZs)
- Use load balancer (ALB/NLB) to distribute traffic
- Use Auto Scaling Group for compute
- Use RDS Multi-AZ for database failover
- Use S3 for static assets (11 nines durability)
- Use CloudFront CDN for global distribution
- Use Route 53 for DNS failover

9. Q: What's CloudFormation and why use it?

A: CloudFormation is Infrastructure as Code (IaC) service that models and provisions AWS resources using templates. Benefits: version-controlled infrastructure, repeatable deployments, consistent environments, rollback on failure, dependency management.

10. Q: What's the difference between ALB and NLB?
A: ALB is Layer 7 (HTTP/HTTPS) with path-based routing, sticky sessions. NLB is Layer 4 (TCP/UDP) with lower latency, static IP. Use ALB for web apps, APIs. Use NLB for high-performance TCP/UDP, gaming.

[Page 27]
Chapter 28: System Design with AWS
Design a URL Shortener (bit.ly)
Requirements:
- Convert long URL to short code (bit.ly/abc123)
- Redirect to original URL when accessed
- Handle millions of URLs
AWS Architecture:



```
|                                                                                |
|  Why this design:                                                              |
 |  - API Gateway + Lambda: Serverless, no idle cost                            |
 |  - DynamoDB: High throughput, low latency key-value lookups                  |
 |  - CloudFront: Cache redirects for popular URLs (reduces DynamoDB calls)     |
 |  - Route 53: DNS for custom domain                                           |
|                                                                                +
 +--------------------------------------------------------------------------------+
```

Chapter 29: Troubleshooting Scenarios

Scenario 1: EC2 Instance Unreachable

Symptoms: Cannot SSH into EC2 instance.

Troubleshooting Steps:

1. Check Security Group inbound rules (SSH port 22 open, source IP correct)
2. Check NACL rules (inbound/outbound for port 22)
3. Check instance status (running, not terminated)
4. Check route table (instance has route to internet)
5. Check internet gateway (attached to VPC)
 6. Check system log (EC2 console -> Actions -> Instance Settings -> Get System Log)

Scenario 2: S3 Access Denied

Symptoms: Cannot access S3 bucket (AccessDenied error).

Troubleshooting Steps:

1. Check IAM policy for your user/role
2. Check bucket policy (allows your principal)
3. Check bucket ACLs (if using)
4. Check bucket public access block settings
5. Check object ownership (ACL disabled?)
6. Check encryption settings (KMS key permissions)

Scenario 3: Lambda Function Timeout

Symptoms: Lambda function times out after 3 seconds.

Troubleshooting Steps:

1. Check Lambda timeout setting (default 3 seconds, increase if needed)
2. Check external API calls (are they hanging?)
3. Check database connection (is it established?)
4. Check for infinite loops
5. Check memory (low memory = slower execution)
6. Check CloudWatch logs for errors

Scenario 4: RDS Connection Issues

Symptoms: Application cannot connect to RDS.

Troubleshooting Steps:

1. Check RDS security group (allow inbound from app on port 3306/5432)
2. Check RDS is in same VPC as application
3. Check RDS is publicly accessible (if from internet)
4. Check username/password (Secrets Manager)
5. Check RDS instance status (available)
6. Check max connections limit

[Page 28]

CONCLUSION

Key Takeaways

1. **AWS is vast but learnable:** Focus on core services (EC2, S3, Lambda, RDS, VPC, IAM)

2. **Choose the right tool:** EC2 for control, Lambda for serverless, ECS for containers

3. **Design for failure:** Use Multi-AZ, load balancers, auto scaling

4. **Security is layered:** IAM, Security Groups, NACLs, encryption

5. **Observability is essential:** CloudWatch metrics, logs, alarms

6. **Infrastructure as Code:** CloudFormation or CDK for repeatable infrastructure

Your Interview Preparation Checklist

- [ ] Understand EC2, S3, Lambda, RDS, DynamoDB
- [ ] Know VPC, subnets, security groups, NACLs
- [ ] Understand load balancers (ALB vs NLB)
- [ ] Know IAM (users, groups, roles, policies)
- [ ] Understand CI/CD (CodePipeline, CodeBuild, CodeDeploy)
- [ ] Compare services (EC2 vs Lambda, S3 vs EBS vs EFS, RDS vs DynamoDB)
- [ ] Practice system design with AWS
- [ ] Review troubleshooting scenarios

Final Tips

1. **Draw diagrams:** Visualize architectures during interviews

2. **Compare services:** Show you understand trade-offs

3. **Start simple:** Master core services first

4. **Practice hands-on:** Build actual projects

5. **Stay updated:** AWS adds new services frequently

Recommended Certifications

Certification	**Level**	**Focus**
---	---	---
AWS Cloud Practitioner	Foundational	AWS basics
AWS Solutions Architect Associate	Associate	Architecture design
AWS Developer Associate	Associate	Application development
AWS DevOps Engineer Professional	Professional	CI/CD, operations

Good luck with your interview!

About This Book

This book is designed for developers and DevOps engineers preparing for AWS interviews. Every concept includes real scenarios, diagrams, and tool comparisons. The goal is to prepare you for both technical interviews and real-world AWS engineering.

May your AWS journey be successful and your interviews outstanding.

[End of Both Books]